

Desarrollo e implementación de una plataforma web para la gestión de un taller de mecánico

CAMP/
INSTITUTO [IES CAMP DE MORVEDRE]

TALLER MECÁNICO (ASIR)

PROYECTO FINAL DE CICLO
ADMINISTRACIÓN DE SISTEMAS EN RED



AUTOR:
ANTONIO CORREDERA CUBELLS

TUTOR:
FRANCISCO ALFONSO, HUESO PASTOR

FECHA:
2026

PROYECTO FINAL DE CICLO

Implantación y monitorización de un sistema web

para la monitorización de páginas web y la gestión de un taller mecánico

CFGS Administración de Sistemas Informáticos en Red

Autor: Corredera Cubells, Antonio

Curso: 2025/2026

Centro docente: IES Camp de Morvedre

Tutor/a individual: Hueso Pastor, Francisco Alfonso

La memoria documenta el análisis, diseño, implantación y validación de una solución web orientada a la gestión de un taller mecánico. El trabajo se ha enfocado desde la perspectiva del ciclo de Administración de Sistemas Informáticos en Red, por lo que no se limita a describir la aplicación, sino que explica también la infraestructura, la seguridad, la observabilidad, el despliegue con contenedores y la propuesta de un entorno cloud.

ÍNDICE

ÍNDICE DE FIGURAS Y TABLAS	5
Índice de figuras	5
Índice de tablas	5
1. INTRODUCCIÓN	7
1.1. Módulos del CFGS ASIR a los que implica	7
1.2. Breve descripción del PFC	7
Objetivos del PFC	8
2. ESTUDIO PREVIO	9
2.1. Situación de partida	9
2.2. Necesidades detectadas	9
2.3. Soluciones existentes y solución propuesta	10
3. PLAN DE TRABAJO	11
4. DISEÑO	12
4.1. Arquitectura general	12
4.2. Modelo de datos	12
4.3. Flujo de reserva de cita	13
4.4. Diseño de redes y despliegue cloud	14
5. IMPLANTACIÓN	16
5.1. Aplicación web	16
5.2. Seguridad de aplicación	16
5.3. Docker Compose local	17
5.4. despliegue en AWS	17
Reglas de entrada necesarias para abrir las UIs desde navegador	26
5.5. Monitorización y alertas	27
5.6. Simulador de tráfico	27
5.7. Requisitos no funcionales y criterios de calidad	30

<u>5.8. Detalle de módulos funcionales</u>	31
<u>5.8.1. Módulo de autenticación y usuarios</u>	31
<u>5.8.2. Módulo de citas</u>	32
<u>5.8.3. Módulo de contenidos: noticias y consejos</u>	32
<u>5.8.4. Módulo de administración</u>	33
<u>5.9. Procedimiento de despliegue detallado</u>	33
<u>5.9.1. Seguimiento del primer despliegue en EC2</u>	34
<u>5.10. Copias de seguridad y restauración</u>	36
<u>5.11. Despliegues alternativos: Coolify y Dokploy</u>	36
<u>5.12. Operación diaria y mantenimiento</u>	37
<u>6. RECURSOS Y PRESUPUESTO</u>	39
<u>6.1. Recursos utilizados</u>	39
<u>6.2. Presupuesto anual orientativo</u>	39
<u>7. VALIDACIÓN Y PRUEBAS</u>	40

ÍNDICE DE FIGURAS Y TABLAS

Índice de figuras

Figura 1. Arquitectura general del sistema web y servicios de observabilidad.

Figura 2. Modelo de datos principal de la aplicación.

Figura 3. Flujo de reserva de cita desde la interfaz web.

Figura 4. Despliegue recomendado en AWS con EC2 y Docker Compose.

Figura 5. Consola AWS: opción «Lanzar instancia» en el laboratorio.

Figura 6. Asignación de nombre e imagen AMI (Amazon Linux).

Figura 7. Selección del tipo de instancia (t3.small).

Figura 8. Creación de par de claves (.pem) para acceso SSH.

Figura 9. Diálogo de creación del par de claves.

Figura 10. Configuración de almacenamiento (≥ 30 GB).

Figura 11. Detalles avanzados: datos de usuario (ec2-user-data-bootstrap.sh).

Figura 12. Lanzamiento de la instancia y acceso al panel EC2.

Figura 13. Detalle de instancia: pestaña Seguridad y grupo asociado.

Figura 14. Acceso al identificador del grupo de seguridad.

Figura 15. Edición de reglas de entrada del Security Group.

Figura 16. Reglas de entrada para UIs de monitorización y servicios.

Figura 17. Asistente «Conectar» a la instancia EC2.

Figura 18. Salida del log taller-ec2-bootstrap.log (bootstrap).

Figura 19. Consulta de credenciales Grafana en .env (grep GRAFANA).

Figura 20. Estado de contenedores (docker ps).

Figura 21. Acceso a la aplicación web mediante IPv4 pública.

Figura 22. Flujo de monitorización y simulación de tráfico.

Figura 23. Flujo operativo del despliegue EC2 y consulta de credenciales.

Figura 24. Funcionamiento del simulador Apache JMeter dentro del proyecto.

Índice de tablas

Tabla 1. Relación entre módulos del CFGS ASIR y contenidos aplicados.

Tabla 2. Requisitos funcionales del sistema.

Tabla 3. Comparativa de soluciones existentes.

Tabla 4. Plan de trabajo y temporalización.

Tabla 5. Entidades principales de la base de datos.

Tabla 6. Servicios del despliegue Docker local.

Tabla 7. Servicios y puertos en despliegue AWS.

Tabla 8. Métricas y alertas implantadas.

Tabla 9. Requisitos no funcionales.

Tabla 10. Checklist de despliegue.

Tabla 11. Grado de consecución de objetivos.

Tabla 12. Problemas encontrados y resolución.

Tabla 13. Recursos utilizados.

Tabla 14. Presupuesto anual orientativo.

Tabla 15. Matriz de pruebas funcionales y técnicas.

Tabla 16. Guion de demostración.

Tabla 17. Grado de consecución de objetivos.

Tabla 18. Problemas encontrados y resolución.

Tabla 19. Propuestas de mejora.

Tabla 20. Inventario de artefactos del repositorio.

1. INTRODUCCIÓN

1.1. Módulos del CFGS ASIR a los que implica

El proyecto integra competencias de varios módulos del ciclo formativo de ASIR. La solución desarrollada parte de una aplicación web de gestión, pero su valor académico reside en la implantación completa de un servicio: definición de base de datos, publicación mediante Apache y PHP, despliegue con Docker, operación en cloud, seguridad básica, monitorización técnica y pruebas de carga controladas.

Tabla 1. Relación entre módulos del CFGS ASIR y contenidos aplicados

Fuente: elaboración propia.

1.2. Breve descripción del PFC

El PFC consiste en la implantación de un sistema web para un taller mecánico que permite organizar la relación entre clientes, citas, contenidos informativos, administración interna y monitorización. El sistema tiene tres perfiles principales: visitantes, usuarios registrados y administradores. Los visitantes pueden consultar información pública y solicitar una cita como invitados; los usuarios registrados pueden gestionar sus datos y consultar su historial; los administradores disponen de un panel para gestionar usuarios, agenda, noticias y consejos.

La solución está desarrollada con PHP, MySQL, HTML5, CSS3 y JavaScript. El acceso a datos se realiza mediante PHP Data Objects y consultas preparadas. La autenticación utiliza sesiones PHP, regeneración de identificador de sesión y contraseñas almacenadas con el algoritmo bcrypt. Las operaciones de administración quedan restringidas por rol, lo que separa las acciones de usuario normal de las funciones de gestión del negocio.

La parte más relevante es la implantación completa del servicio. El repositorio de GitHub en el que se basa nuestro proyecto incluye Dockerfile, varios ficheros Docker Compose, guías de despliegue local, despliegue AWS, despliegue Coolify/Dokploy, monitorización Prometheus/Grafana, Alertmanager, exporters, blackbox y un simulador de tráfico separado de la

aplicación principal. Esto nos permite demostrar no solo que la aplicación funciona, sino que puede operarse y vigilarse como un servicio real.

Todo esto se plantea para una pequeña empresa de reparación de vehículos que necesita abandonar procesos manuales de reserva y comunicación. La aplicación centraliza citas y publicaciones, reduce la necesidad de llamadas telefónicas, teniendo acceso desde cualquier navegador y aporta indicadores para anticipar problemas de disponibilidad o rendimiento.

Objetivos del PFC

- Diseñar e implantar una aplicación web funcional para la gestión básica de un taller mecánico.
- Definir una base de datos relacional que mantenga usuarios, credenciales, citas, noticias y consejos.
- Publicar la solución mediante contenedores Docker para que el despliegue sea reproducible en local y en cloud.
- Añadir una capa de monitorización que permita observar el sistema, uso de la aplicación, base de datos y métricas de negocio.
- Documentar una propuesta de despliegue profesional en AWS con costes, seguridad perimetral y la estrategia de mantenimiento.

Tabla 2. Requisitos funcionales del sistema

Fuente: elaboración propia.

2. ESTUDIO PREVIO

2.1. Situación de partida

El escenario propuesto corresponde a un taller mecánico de tamaño pequeño o mediano que atiende tanto reparaciones planificadas como incidencias puntuales. En un funcionamiento tradicional, las reservas se realizan por teléfono, mensajes o presencialmente. Este modelo es suficiente en fases iniciales, pero genera problemas cuando aumenta el número de clientes: se duplican horarios, pérdida de información, poca trazabilidad y dependencia de una persona encargada de gestionar las citas y horarios.

La empresa también necesita publicar información de interés para sus clientes. Las noticias, campañas, avisos de mantenimiento y consejos técnicos suelen difundirse por canales externos o de forma informal y esto dispersa la información. Una aplicación propia permite centralizar los contenidos y nos aporta una visión más profesional hacia el cliente.

Desde el punto de vista de sistemas, el taller no solo necesita una página web, sino un servicio estable. Si se produce una caída, un aumento de latencia o una saturación de base de datos, el negocio puede perder reservas. Por este motivo el proyecto incorpora mecanismos de observabilidad desde el inicio, con métricas, dashboards, alertas y pruebas de carga controladas.

2.2. Necesidades detectadas

La primera necesidad es digitalizar el proceso de agendar una cita. El sistema debe permitirnos seleccionar fecha y hora, guardar el motivo de la visita y evitar que dos clientes ocupen la misma hora. Además, debe admitir reservas de usuarios autenticados y reservas de invitados, ya que no todos los clientes quieren registrarse antes de pedir una cita.

La segunda necesidad es la de separar responsabilidades. El cliente solo debe acceder a su información y a sus citas, mientras que el administrador necesita una vista completa para gestionar agenda, usuarios y publicaciones. Esta separación se realiza con roles, sesiones y páginas protegidas.

La tercera necesidad es que el despliegue sea sencillo. Un proyecto de ASIR debe poder instalarse de forma repetible, por lo que Docker Compose es una solución adecuada: define servicios, redes, volúmenes, puertos, variables de entorno y dependencias entre contenedores.

La cuarta y última necesidad es gestionar la aplicación mediante un sistema que nos permita ver qué ocurre en su interior en todo momento. Prometheus recoge métricas, Grafana las muestra en paneles, Alertmanager permite notificaciones y Blackbox comprueba rutas HTTP desde fuera del proceso de la aplicación. Con ello se detectan problemas antes de que afecten al cliente final.

2.3. Soluciones existentes y solución propuesta

Tabla 3. Comparativa de soluciones existentes

Fuente: elaboración propia.

Se elige una solución propia porque la aplicación no depende de un proveedor externo, utiliza tecnologías conocidas en entornos LAMP y se acompaña de infraestructura de contenedores, documentación, métricas y una propuesta cloud. Esta decisión es coherente con el objetivo formativo: mostrar que el alumno es capaz de diseñar, desplegar y mantener un servicio completo.

Dentro de la solución he elegido las tecnologías descritas porque son software libre, estándares de facto (LAMP, Prometheus, Docker) y otras alternativas que podríamos considerar serían PostgreSQL, Nagios/Zabbix, WordPress, XAMPP frente a Docker Compose.

3. PLAN DE TRABAJO

La planificación se ha distribuido en las siguientes fases: análisis, diseño, construcción, despliegue, monitorización, pruebas y documentación.

Lógicamente las horas son estimatorias y se ajustan a las horas del módulo de Proyecto, aunque el desarrollo real del repositorio contiene más detalles que los recogidos de forma resumida en esta tabla.

Tabla 4. Plan de trabajo y temporalización

Fuente: elaboración propia.

4. DISEÑO

4.1. Arquitectura general

La arquitectura se organiza alrededor de una aplicación PHP ejecutada en Apache, una base de datos MySQL y un conjunto de servicios auxiliares de monitorización. Entorno desarrollo, Docker Compose levanta todos los componentes en una misma máquina. El compose en concreto la nube de AWS , es nuestro entorno de producción publica la aplicación web y también puede publicar las interfaces de Grafana, Prometheus, Alertmanager y la UI del simulador para acceder desde el navegador. Esa publicación depende de dos capas: el bind de Docker, configurado para escuchar en todas las interfaces, y las reglas de entrada del Security Group, que deben limitarse a la IP o a la red administrativa siempre que sea posible.

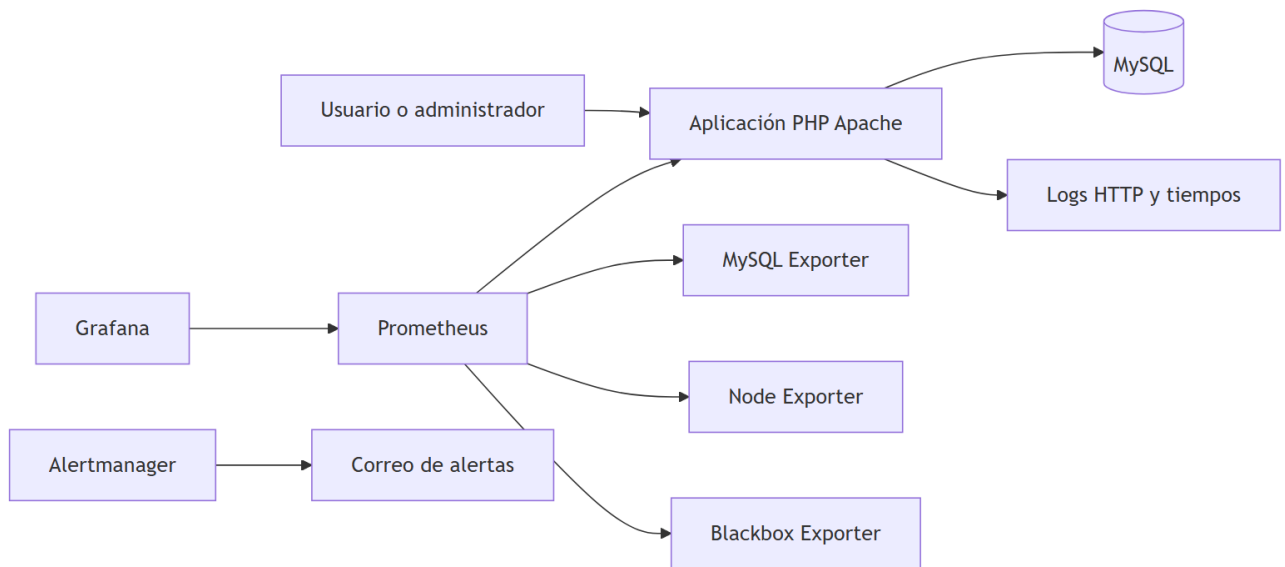


Figura 1. Arquitectura general del sistema web y servicios de observabilidad.

Fuente: elaboración propia a partir del repositorio del proyecto.

4.2. Modelo de datos

La base de datos se denomina “trabajo_final_php” y utiliza MySQL con tablas InnoDB. El modelo separa datos personales y credenciales para evitar mezclar información de perfil con autenticación. Las citas pueden asociarse a un usuario registrado o a un invitado. Las noticias y consejos están vinculados a usuarios administradores mediante claves foráneas.

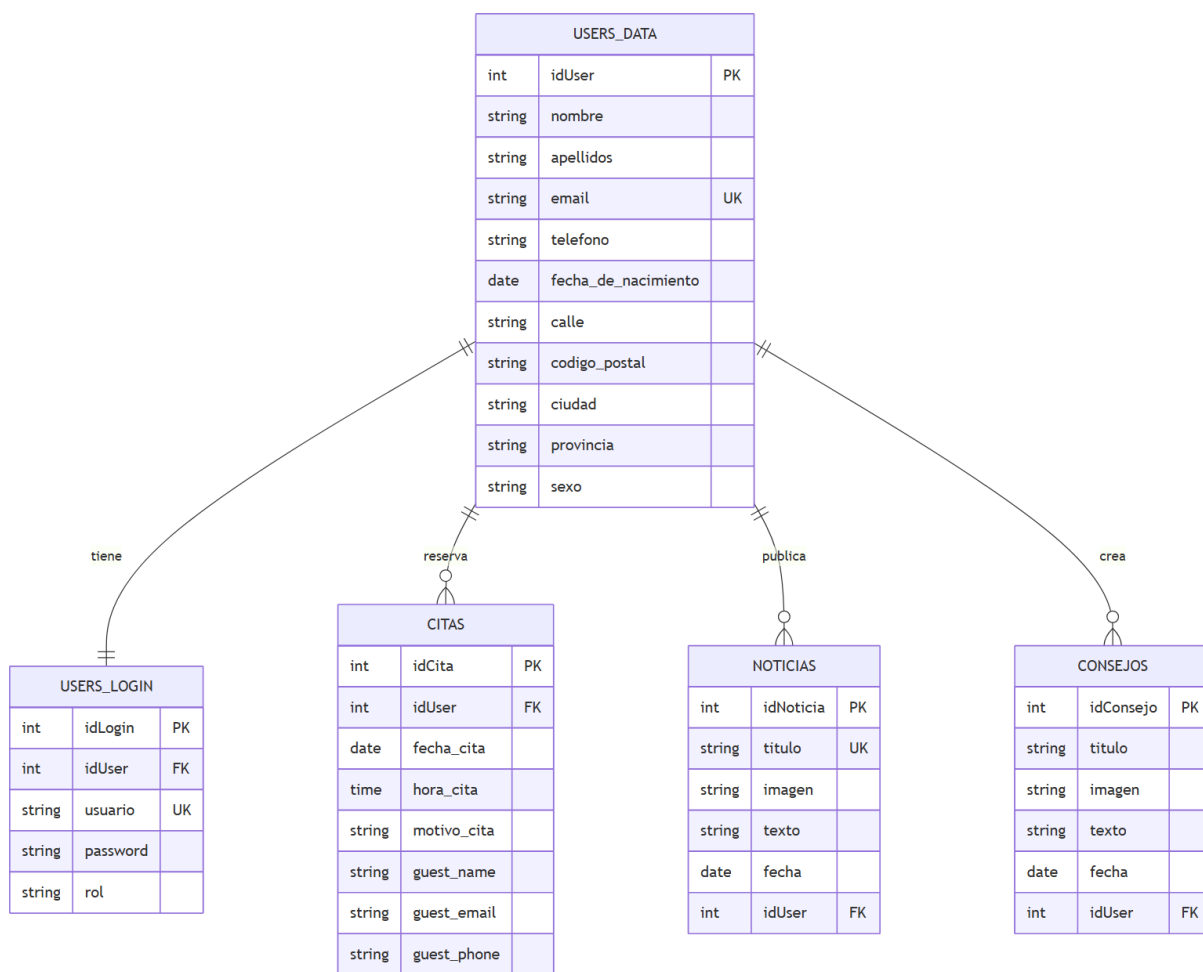


Figura 2. Modelo de datos principal de la aplicación.

Fuente: elaboración propia a partir del repositorio del proyecto.

Tabla 5. Entidades principales de la base de datos

Fuente: elaboración propia.

4.3. Caso de uso: reserva de cita

El flujo de reserva se diseñó para ser sencillo. El usuario selecciona una fecha, la interfaz consulta la API para recuperar horarios ocupados y solo permite confirmar los tramos disponibles. Si el cliente está autenticado, la cita se vincula a su identificador; si no lo está, se exigen los datos básicos de contacto. La API valida formato de fecha y hora, comprueba que la cita no esté en el pasado y evita duplicados por fecha y hora.

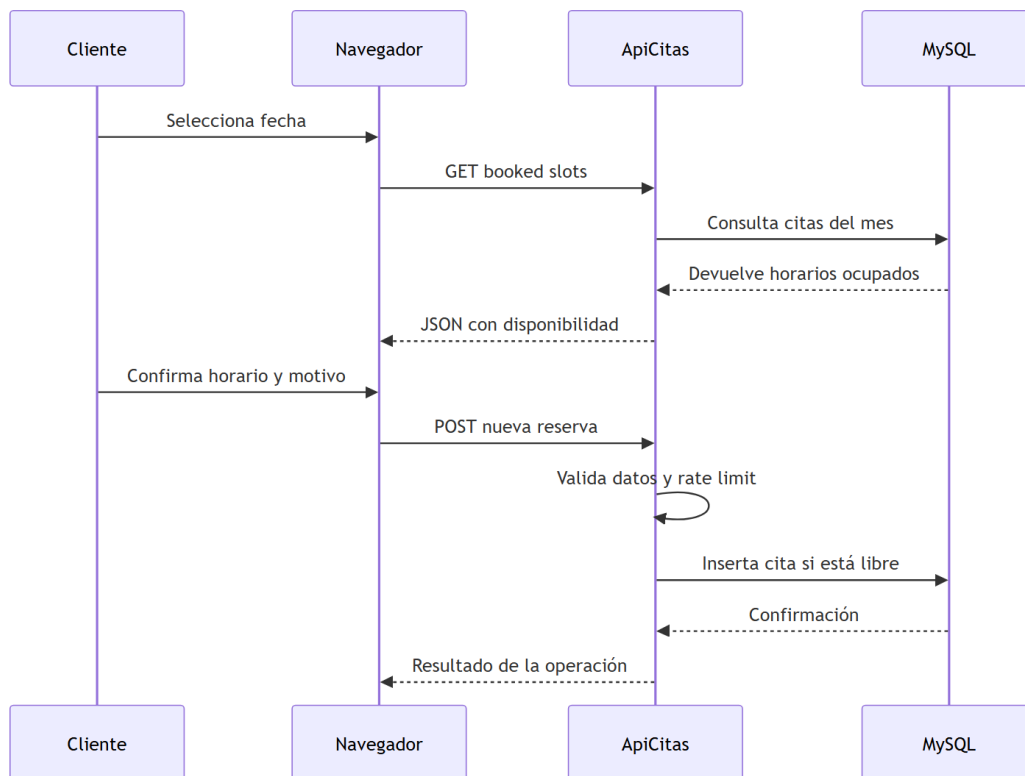


Figura 3. Flujo de reserva de cita desde la interfaz web.

Fuente: elaboración propia a partir del repositorio del proyecto.

4.4. Diseño de redes y despliegue cloud

El diseño Docker local separa los servicios en dos áreas de red: la red de aplicación y la red de monitorización ya que así conseguimos en mayor orden y nos aporta seguridad ya que el servicio web se conecta a ambas porque necesita hablar con MySQL y a la vez, exponer métricas a Prometheus. MySQL no debe exponerse públicamente en producción. En AWS, el compose recomendado publica la aplicación web en el puerto 80 lo cual no es recomendable, pero lo hemos utilizado ya que estamos en un entorno de laboratorio y además publica las interfaces de monitorización en puertos concretos para que puedan abrirse desde el navegador cuando lo permita el Security Group. Los exporters técnicos, como cAdvisor y Telegraf, funcionan únicamente a través de 127.0.0.1 o a la red interna, porque solo los debe consultar Prometheus.

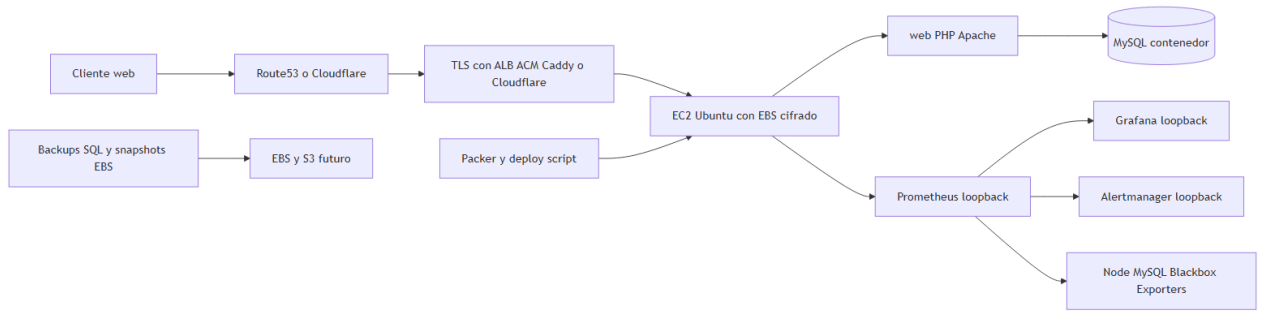


Figura 4. Despliegue recomendado en AWS con EC2 y Docker Compose.

Fuente: elaboración propia a partir del repositorio del proyecto.

5. IMPLANTACIÓN

5.1. Aplicación web

La aplicación se estructura en páginas PHP: index.php (inicio), login.php, registro.php, perfil.php, citas.php (citación y calendario), noticias.php, consejo.php y, para el personal del taller, el panel de admin/ (admin/index.php y módulos CRUD en admin/usuarios.php, admin/citas.php, admin/noticias.php y admin/consejos.php). Los archivos comunes de cabecera, pie y funciones auxiliares se ubican en includes/, mientras que la conexión a base de datos se centraliza en config/database.php. Esta separación reduce la duplicación y facilita realizar cambios en configuración.

El registro crea una entrada en users_data y otra en users_login dentro de una transacción. Antes de insertar, valida los campos obligatorios, formato de correo, teléfono español y código postal. La contraseña se guarda mediante password_hash, evitando texto plano. El login usa password_verify, regenera el identificador de sesión y genera un nuevo token CSRF.

La gestión de citas se apoya en una interfaz con calendario y llamadas fetch a api/citas_api.php. La API responde con JSON y registra métricas de cada petición mediante includes/metrics_logger.php, así las peticiones relacionadas con la cita también quedan reflejadas en Prometheus.

La administración incluye CRUD de usuarios, citas, noticias y consejos en las rutas admin/usuarios.php, admin/citas.php, admin/noticias.php y admin/consejos.php (panel resumido en admin/index.php). Las noticias permiten subir imágenes con validación de tipo MIME, extensión, tamaño máximo y bytes mágicos. Los consejos técnicos pueden cargarse desde datos semilla y editarse desde el mismo panel de admin/.

5.2. Seguridad de aplicación

La autenticación se basa en sesiones PHP ya que incorpora en dicha sesión el perfil del usuario autenticado. Las páginas comprueban la existencia de user_id y las zonas de administración verifican el rol admin. Esta lógica evita que un usuario normal acceda a funciones de administrador.

La protección frente a inyección SQL se implementa con PDO y sentencias preparadas. El uso de `PDO::ATTR_EMULATE_PREPARES => false` favorece prepared statements nativos del motor. Los datos de salida se tratan con `htmlspecialchars` y los datos de entrada se limpian con `trim` y `strip_tags` en funciones auxiliares.

El proyecto incorpora tokens CSRF en formularios sensibles como login y registro. En la API de citas se añade una limitación de frecuencia para reducir spam de reservas. En producción se recomienda complementar estas medidas con HTTPS obligatorio, cookies seguras, cabeceras de seguridad y políticas de retención de datos personales.

5.3. Docker Compose local

El despliegue local se define en `docker-compose.yml`. Este fichero crea la aplicación web, MySQL, Prometheus, Alertmanager, Grafana, Node Exporter, MySQL Exporter, cAdvisor, Telegraf, blackbox-exporter y, opcionalmente, el perfil traffic con simulador y UI. Los volúmenes persistentes mantienen datos de MySQL, Prometheus, Grafana y Alertmanager, además de imágenes subidas y logs de la aplicación.

Tabla 6. Servicios del despliegue Docker local

Fuente: elaboración propia.

5.4. Despliegue en AWS

Para producción se propone usar una instancia EC2 con Docker Compose y volúmenes persistentes. El fichero `docker-compose.aws.yml` evita montar todo el código como bind-mount, como se nombró anteriormente no publica MySQL y separa los servicios por perfiles: web y mysql como base, monitoring para Prometheus/Grafana/Alertmanager/exporters, y traffic para Apache JMeter y su UI. En la configuración AWS actual, las UIs de monitorización se publican para navegador mediante `MONITORING_UI_HOST_BIND=0.0.0.0`; por eso la protección real debe estar en el Security Group, usando `MONITORING_SG_CIDR` con la IP administrativa en vez de abrir `0.0.0.0/0` en producción.

El despliegue en AWS está totalmente automatizado apoyandose en Packer. Este se encarga de construir una imagen de máquina de amazon (AMI) para base con Docker Engine y plugin Compose, y en “scripts/ec2-user-data-bootstrap.sh” para Amazon Linux 2023. El bootstrap instala Docker, Compose y Buildx, crea swap, clona el repositorio en “/opt/taller_mecanico_asir”, genera secretos cuando detecta valores de ejemplo y ejecuta scripts/deploy_aws_docker.sh. El script de despliegue realiza preflight de rutas, memoria y disco, backup opcional de MySQL, build/pull/up del compose, comprobaciones de salud y smoke tests de HTTP, Grafana, Prometheus, Alertmanager y JMeter cuando el perfil traffic está activo.

5.4.1. Lanzamiento de una instancia EC2

1. Una vez accedemos al laboratorio nos dirigiremos a la opción de “Lanzar la instancia”.

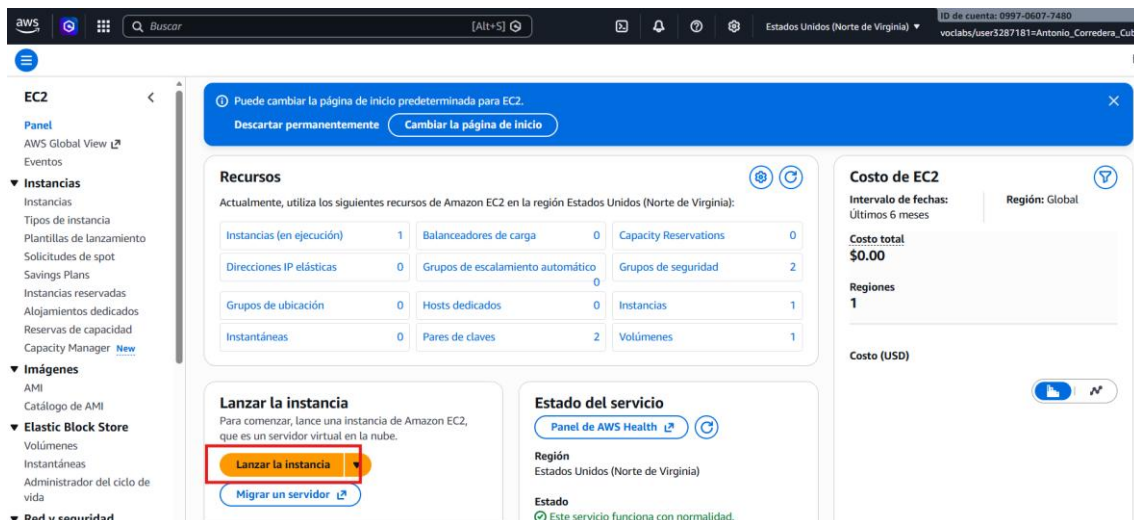


Figura 5. Consola AWS: opción «Lanzar instancia» en el laboratorio.

2. Elegiremos el nombre que queramos que tenga y elegiremos la imagen que queramos utilizar, en nuestro caso seleccionaremos “Amazon Linux”.

3. El tipo de instancia seleccionaremos t3.small ya que en la capa gratuita es la que más nos conviene gracias a sus 2GB de memoria RAM.

Figura 7. Selección del tipo de instancia (t3.small).

4. Crearemos un par de claves para poder conectarnos de forma segura a la instancia que vamos a crear. Pulsamos en “Crear un nuevo par de claves”.

▼ **Par de claves (inicio de sesión)** [Información](#)

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - *obligatorio*

Continuar sin un par de claves (no recomendado) Valor predeterminado ▼ [Crear un nuevo par de claves](#)

Figura 8. Creación de par de claves (.pem) para acceso SSH.

Se nos abrirá el recuadro que vemos a continuación, le daremos un nombre y en nuestro caso hemos seleccionado la opción de .pem ya que nos conectaremos via SSH aunque también podríamos configurarlo para utilizar la famosa herramienta de PuTTY seleccionando .ppk.

Crear par de claves ✕

Nombre del par de claves
Con los pares de claves es posible conectarse a la instancia de forma segura.

ClaveProyecoASIR

El nombre puede incluir hasta 255 caracteres ASCII. No puede incluir espacios al principio ni al final.

Tipo de par de claves

☒ **RSA**
Par de claves pública y privada cifradas mediante RSA

☐ **ED25519**
Par de claves privadas y públicas cifradas ED25519

Formato de archivo de clave privada

☒ **.pem**
Para usar con OpenSSH

☐ **.ppk**
Para usar con PuTTY

⚠ Cuando se le solicite, almacene la clave privada en un lugar seguro y accesible del equipo. **Lo necesitará más adelante para conectarse a la instancia.** [Más información](#) ↗

[Cancelar](#) [Crear par de claves](#)

Figura 9. Diálogo de creación del par de claves.

5. “Configuraciones de red” lo dejaremos por defecto, “Configuración de almacenamiento” Pondremos al menos 30GB.

▼ Configuraciones de red [Información](#)

Editar

Red | [Información](#)

vpc-042d29757dfe82317

Subred | [Información](#)

No hay preferencia (subred predeterminada en cualquier zona de disponibilidad)

Asignación automática de IP pública | [Información](#)

Habilitar

Firewall (grupos de seguridad) | [Información](#)

Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.

☒ Crear un grupo de seguridad

☐ Seleccionar un grupo de seguridad existente

Crearemos un nuevo grupo de seguridad denominado "launch-wizard-1" con las siguientes reglas:

☒ Permitir el tráfico de SSH desde

Ayuda a establecer conexión con la instancia

Cualquier lugar

0.0.0.0/0

☐ Permitir el tráfico de HTTPS desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

☐ Permitir el tráfico de HTTP desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

⚠ Las reglas con origen 0.0.0.0/0 permiten que todas las direcciones IP tengan acceso a la instancia. Le recomendamos que configure las reglas del grupo de seguridad para permitir el acceso únicamente desde direcciones IP conocidas.

×

▼ Configurar almacenamiento [Información](#)

Avanzado

1x 30 GiB gp3 Volumen raíz, 3000 IOPS, No cifrado

Agregar nuevo volumen

🕒 Haga clic en actualizar para ver la información de la copia de seguridad

Las etiquetas que asigne determinan si alguna política de Data Lifecycle Manager realizará una copia de seguridad de la instancia.

🔄

Sistemas de archivos

☐ Archivos de S3: nuevo

☐ EFS

☐ FSx

☒ Ninguno

Figura 10. Configuración de almacenamiento (≥ 30 GB).

6. Nos dirigimos a “Detalles avanzados”. Buscamos “Datos de usuario (opcional)” y elegiremos nuestro archivo “ec2-user-data-bootstrap.sh”. Este nos ayuda a configurar la máquina y a que los contenedores funcionen de forma correcta.

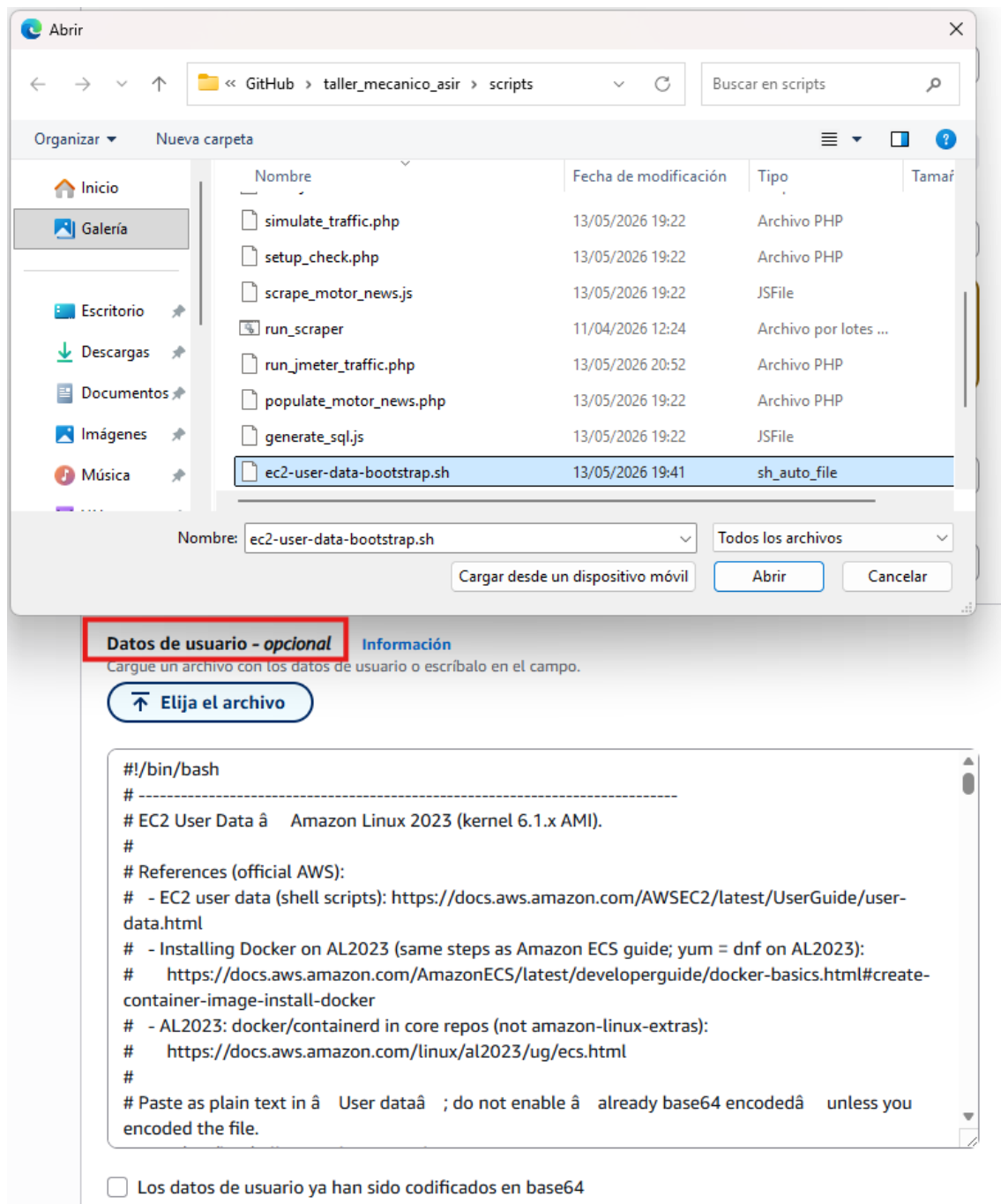


Figura 11. Detalles avanzados: datos de usuario (ec2-user-data-bootstrap.sh).

- Una vez finalizada la configuración ya podemos lanzar la instancia, volvemos al panel donde aparecerá la instancia que hemos creado. Pulsaremos en "ID de la instancia".

☐	Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de	Zona de dispon...
☐	TallerMecanicoASIRTest	i-0a6b3f1b22e5bc293	En ejecución	t3.nano	3/3 comprobaci...	us-east-1c

Figura 12. Lanzamiento de la instancia y acceso al panel EC2.

8. Una vez dentro veremos que aparecen todos los parámetros. A nosotros nos interesa el de Seguridad ya que tendremos que realizar algunos cambios para que Grafana, Prometheus y los demás servicios sean accesibles. Una vez pulsemos en “Seguridad” se desplegarán los detalles de seguridad y pulsaremos en el grupo de seguridad que deseamos modificar.

Resumen de instancia de i-0a6b3f1b22e5bc293 (TallerMecanicoASIRTest) información

Se ha actualizado hace 15 minutos

ID de la instancia
i-0a6b3f1b22e5bc293

Dirección IPv6
-

Tipo de nombre de anfitrión
Nombre de IP: ip-172-31-84-64.ec2.internal

Responder al nombre DNS de recurso privado IPv4 (A)
44.212.74.225 [IP pública]

Dirección IP asignada automáticamente
44.212.74.225 [IP pública]

Rol de IAM
-

IMDSv2
Required

Operador
-

Dirección IPv4 pública
44.212.74.225 | [dirección abierta](#)

Estado de la instancia
En ejecución

Nombre DNS de IP privada (solo IPv4)
ip-172-31-84-64.ec2.internal

Tipo de instancia
t3.nano

ID de VPC
vpc-042d29757dfe82317

ID de subred
subnet-0a01a6ad3e85cacec

ARN de instancia
arn:aws:ec2:us-east-1:099706077480:instance/i-0a6b3f1b22e5bc293

Direcciones IPv4 privadas
172.31.84.64

DNS público
ec2-44-212-74-225.compute-1.amazonaws.com | [dirección abierta](#)

Direcciones IP elásticas
-

Hallazgo de AWS Compute Optimizer
[Suscribirse a AWS Compute Optimizer para recibir recomendaciones.](#) | [Más información](#)

Nombre del grupo de Auto Scaling
-

Administradas
falso

Detalles | Estado y alarmas | Monitoreo | **Seguridad** | Redes | Almacenamiento | Etiquetas

▼ Detalles de seguridad

Rol de IAM
-

Grupos de seguridad
sg-0044e77b81892fe77 (launch-wizard-1)

ID del propietario
099706077480

Hora de lanzamiento
Sun May 17 2026 16:24:09 GMT+0200 (hora de verano de Europa central)

▼ Reglas de entrada

Nombre	ID de la regla del grupo d...	Intervalo de pu...	Protocolo	Origen	Grupos de seguridad	Descripción
-	sg-04b8953ca3afaf4cd	22	TCP	0.0.0.0/0	launch-wizard-1	-

Figura 13. Detalle de instancia: pestaña Seguridad y grupo asociado.

9. Pulsamos en el id de seguridad para poder editar las reglas que vamos a agregar.

☐	Name	ID de grupo de seguridad	Nombre del grupo de seguridad	ID de la VPC
☐	-	sg-0044e77b81892fe77	launch-wizard-1	vpc-042d29757dfe82317

Figura 14. Acceso al identificador del grupo de seguridad.

10. Una vez dentro entramos en la opción de “Editar reglas de entrada”.

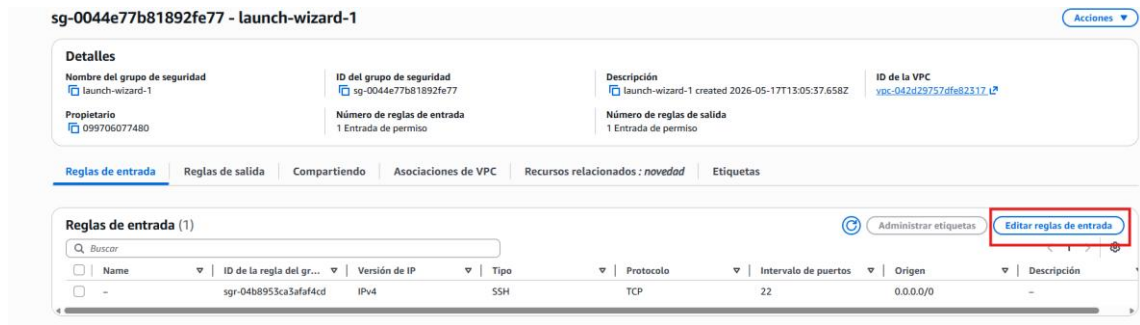


Figura 15. Edición de reglas de entrada del Security Group.

11. En nuestro caso para que funcionen todos los servicios necesarios añadiremos las siguientes reglas.

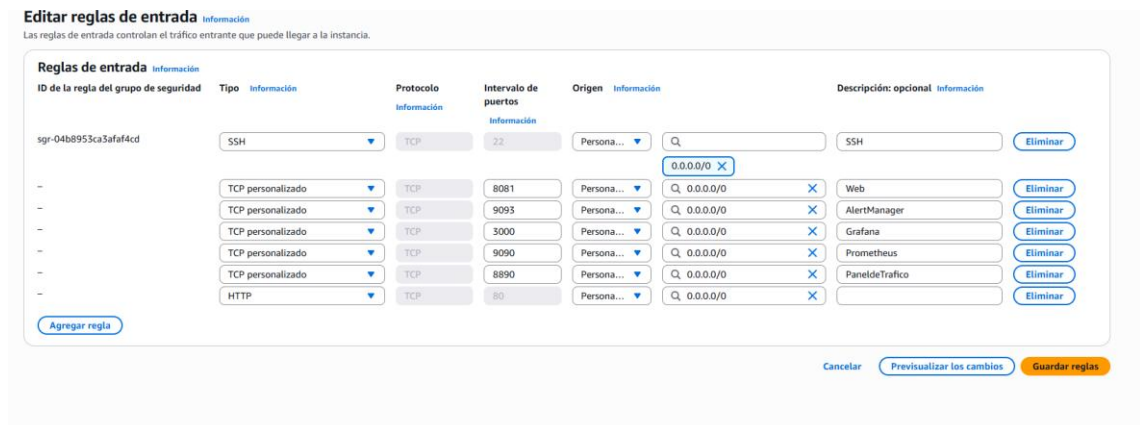


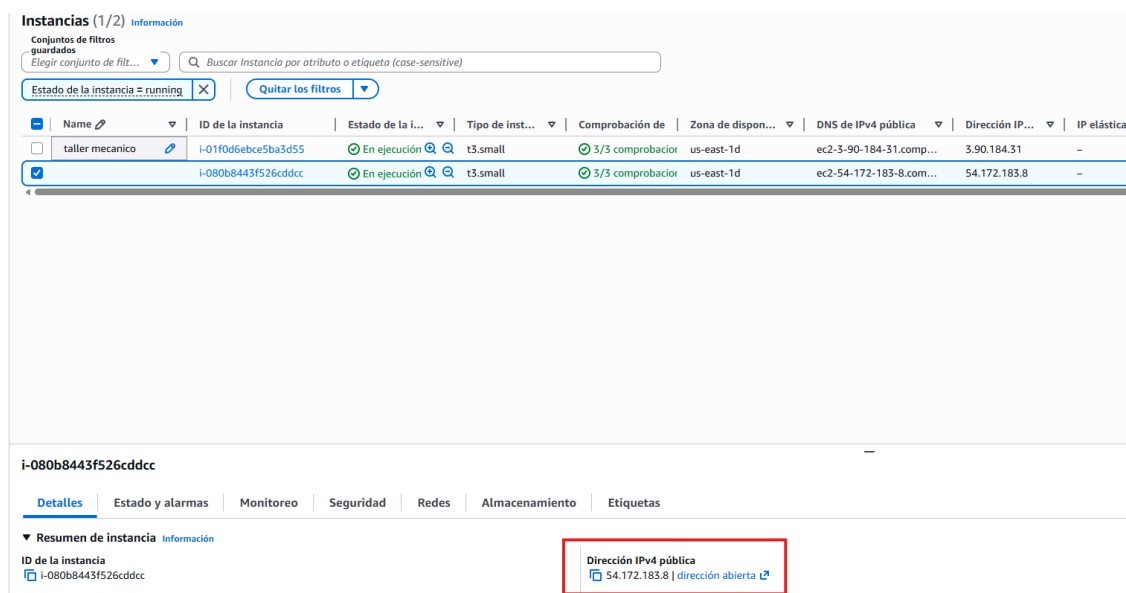
Figura 16. Reglas de entrada para UIs de monitorización y servicios.

12. Una vez ya tengamos las reglas guardadas volvemos al menu de instancias de EC2, seleccionaremos la instancia y pulsaremos el boton de “Conectar”.



Figura 17. Asistente «Conectar» a la instancia EC2.

13. Una vez arranque la máquina utilizaremos el comando “cat /var/log/taller-ec2-bootstrap.log” este nos permite visualizar el archivo “taller-ec2-bootstrap.log”, el cual nos muestra como se esta construyendo toda nuestra estructura, es decir vemos como se instalan los programas, la configuración de las variables de entorno y si ha



Instancias (1/2) Información

Conjuntos de filtros guardados
Elegir conjunto de filt...

Buscar instancia por atributo o etiqueta (case-sensitive)

Estado de la instancia $\neq$ running X Quitar los filtros

	Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de	Zona de dispon...	DNS de IPv4 pública	Dirección IP...	IP elástica
☐	taller mecanico	i-01f0d6ebce5ba3d55	En ejecución	t3.small	3/3 comprobador	us-east-1d	ec2-3-90-184-31.comp...	3.90.184.31	-
☒		i-080b8443f526cddcc	En ejecución	t3.small	3/3 comprobador	us-east-1d	ec2-54-172-183-8.com...	54.172.183.8	-

i-080b8443f526cddcc

Detalles Estado y alarmas Monitoreo Seguridad Redes Almacenamiento Etiquetas

▼ Resumen de instancia Información

ID de la instancia
i-080b8443f526cddcc

Dirección IPv4 pública
54.172.183.8 | dirección abierta

Figura 21. Acceso a la aplicación web mediante IPv4 pública.

Tabla 7. Servicios y puertos en despliegue AWS

Fuente: elaboración propia.

Reglas de entrada necesarias para abrir las UIs desde navegador

Para poder acceder desde un navegador web a la aplicación desplegada en EC2, docker debe publicar : el puerto host y AWS el grupo de seguridad debe permitir la entrada por el origen previsto. Si una de las dos falla, el servicio puede estar “sano” dentro del servidor pero no abrirse desde el navegador. Los puertos que hemos empleado en esta primera version son: 80 para la aplicación web, 3000 para Grafana, 9090 para Prometheus, 9093 para Alertmanager y 8890 para la UI del simulador JMeter. Estos puertos deben abrirse solo desde la IP del administrador cuando sea posible, por ejemplo con `MONITORING_SG_CIDR=TU_IP/32`.

Por otro lado, para securizar puertos que no se deben abrir a Internet: MySQL 3306, cAdvisor 8080, Telegraf 9273, Node Exporter 9100, MySQL Exporter 9104 y Blackbox Exporter 9115. Son servicios técnicos pensados para Prometheus o para la red interna, no para usuarios finales.

Si Grafana o Prometheus no abren en el navegador, se revisa primero que el despliegue haya terminado, después “docker ps”, después “docker compose ps -a”, y por último las reglas del Security Group. En AWS el propio deploy intenta abrir 3000, 9090, 9093 y 8890 si la instancia tiene permisos IAM y awscli instalado.

5.5. Monitorización y alertas

La observabilidad combina métricas generadas por la aplicación, exporters del sistema y probes sintéticos que nos sirven para validar errores, disponibilidad y latencia . El endpoint metrics.php expone métricas de negocio, salud de la base de datos, peticiones HTTP y tiempos de respuesta. Prometheus las recoge periódicamente, evalúa reglas de alerta y Grafana las presenta en paneles.

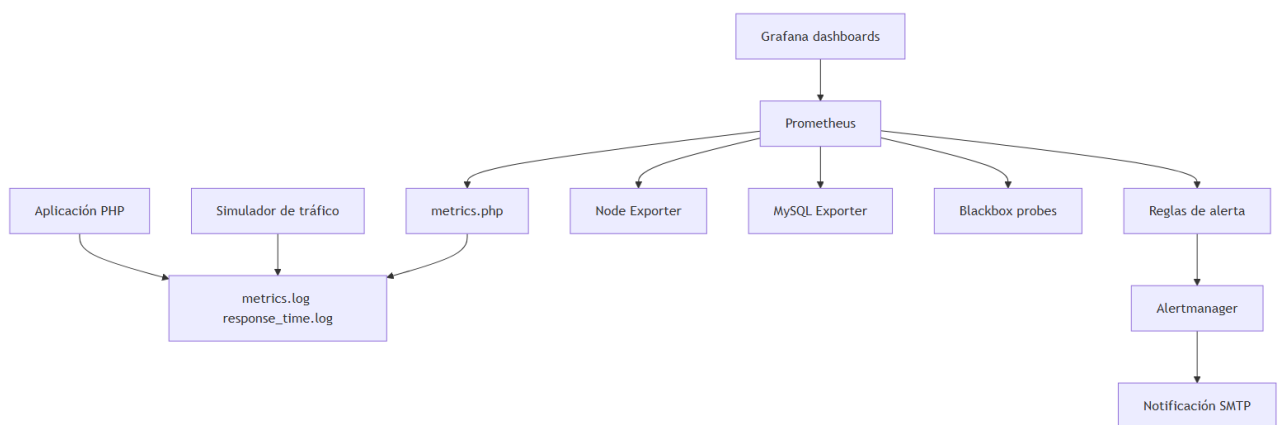


Figura 22. Flujo de monitorización y simulación de tráfico.

Fuente: elaboración propia a partir del repositorio del proyecto.

Tabla 8. Métricas y alertas implantadas

Fuente: elaboración propia.

5.6. Simulador de tráfico

Se ha desplegado una utilidad para realizar simulaciones en la propia infraestructura, en la red que correspondiente al simulador que se ha separado de la aplicación principal para evitar que una herramienta de pruebas forme parte del panel de administración. Con el perfil traffic, Docker Compose inicia un worker con API HTTP interna y una UI independiente. La UI actúa como proxy server-side y no expone el token de control al navegador ya que

cualquiera que tuviera el token podría utilizar el simulador de tráfico a su antojo y podría iniciar pruebas, pararlas y vigilar el tráfico. Las métricas del simulador se escriben en el mismo formato que la aplicación, identificadas con `source=simulator`, lo que permite diferenciarlas en Grafana.

La herramienta incluye controles para evitar abuso: token de control, política de objetivos internos y externos, confirmación para URLs no internas, rechazo de rangos privados salvo configuración de laboratorio y verificación TLS por defecto. Estas medidas son importantes porque una herramienta de carga puede ser peligrosa si se expone sin restricciones.

Procedimiento de uso.

- 1) Acceder a la interfaz web del simulador en el equipo host (puerto 8890 por defecto; variables `TRAFFIC_SIMULATOR_UI_PORT` en desarrollo local con `docker-compose.yml` y `TRAFFIC_SIMULATOR_UI_HOST_PORT` en EC2).
- 2) Destino: indicar la URL base de la prueba (`http://web` dentro de la red Docker, o la URL HTTP/HTTPS pública de la aplicación desplegada).
- 3) Carga: elegir el perfil (Normal, Burst o Idle), el número de usuarios concurrentes y la duración en segundos.
- 4) Ejecución: validar el destino y arrancar la prueba desde la interfaz; el worker ejecuta Apache JMeter en modo CLI y convierte el fichero `results.jtl` al formato de logs compartidos con la aplicación.
- 5) Resultados: revisar el estado de la ejecución, la proporción de respuestas correctas y erróneas y la vista previa de `metrics.log` en la propia interfaz.
- 6) Observabilidad: abrir Grafana (perfil monitoring), el dashboard principal y la fila Simulador para las series con `source=simulator`. El tráfico sintético utiliza el mismo recorrido que el tráfico real (métricas expuestas vía `metrics.php`, Prometheus y Grafana), diferenciado por la etiqueta `source` respecto de `source=app`.
- 7) Informe HTML: al finalizar, si está habilitada la generación del informe, usar el enlace Dashboard JMeter enlazado desde la interfaz (directorio `html-report` bajo el volumen de logs).

Apache JMeter no es una página más del taller. Es una herramienta de pruebas que crea muchas peticiones HTTP para comprobar cómo responde la aplicación cuando hay varios usuarios simulados. En este proyecto se ejecuta en modo CLI, sin interfaz gráfica propia, dentro del contenedor traffic-simulator. Cuando el operador inicia una prueba desde la UI, la UI no llama directamente a JMeter. Primero envía la petición al API interno del worker, protegido por `SIMULATOR_CONTROL_TOKEN`. El worker genera un plan `traffic-test.jmx`, un fichero `routes.csv` con las rutas y pesos, ejecuta `jmeter -n -t traffic-test.jmx -l results.jtl -j jmeter.log`, y guarda también `stdout.log` para diagnóstico. JMeter escribe los resultados en `results.jtl`. El runner del proyecto lee ese fichero y lo convierte al formato que ya entiende `metrics.php`: `logs/metrics.log` recibe líneas como `GET 200 /ruta source=simulator` y `logs/response_time.log` recibe tiempos de respuesta. Así Grafana puede separar el tráfico real de la aplicación, `source=app`, del tráfico de prueba, `source=simulator`.
Uso básico: en local se ejecuta `docker compose --profile traffic up -d` y se abre `http://localhost:8890`. En AWS, el perfil `traffic` viene activado por `COMPOSE_PROFILES=monitoring,traffic` en `.env.aws.example` y la UI se abre en `http://IP_PUBLICA:8890` si el Security Group permite ese puerto.

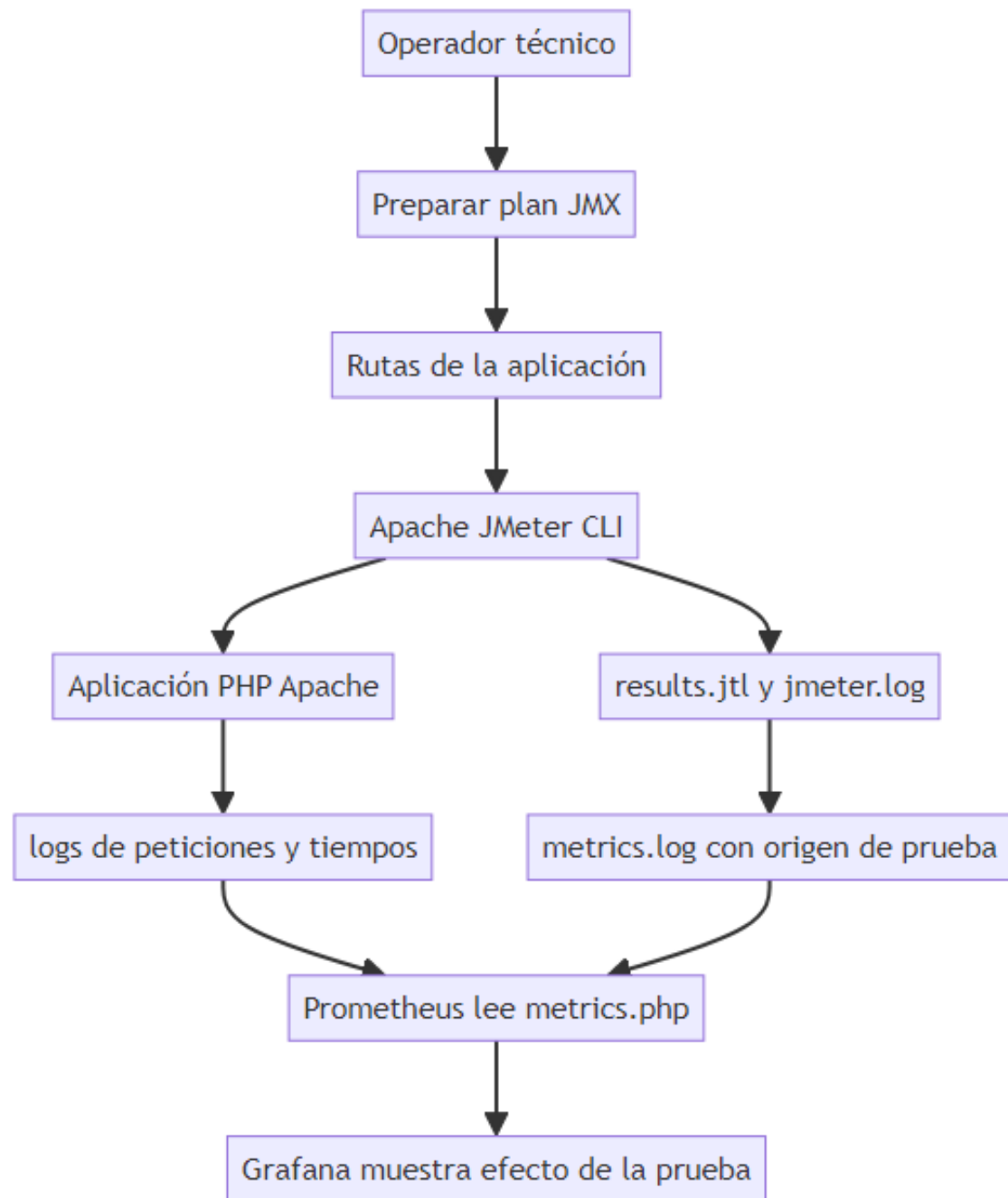


Figura 24. Funcionamiento del simulador Apache JMeter dentro del proyecto.

Fuente: elaboración propia a partir del funcionamiento del contenedor traffic-simulator.

5.7. Requisitos no funcionales y criterios de calidad

Además de las funciones visibles (lo que el usuario puede tocar y manipular), el proyecto se ha diseñado teniendo en cuenta requisitos no funcionales (las reglas que definen como funciona el sistema). En administración de sistemas,

estos requisitos son especialmente importantes porque determinan si una aplicación puede mantenerse en el tiempo, recuperarse ante fallos y adaptarse a los cambios de infraestructura.

La disponibilidad se aborda con healthchecks, separación de servicios y monitorización. El servicio web no debe considerarse operativo hasta que MySQL esté aceptando conexiones. Del mismo modo, la disponibilidad externa no se mide solo por el estado del contenedor, sino mediante probes HTTP que simulan peticiones reales a rutas relevantes en nuestro proyecto.

La fácil gestión o mantenibilidad se apoya en documentación, variables de entorno y ficheros de despliegue separados. El uso de `.env.example`, `.env.aws.example` y varios `compose` evita mezclar configuración de desarrollo con producción. Esta decisión facilita que otro administrador pueda revisar el proyecto y adaptarlo a su entorno sin la necesidad de modificar el código fuente.

La portabilidad se consigue con Docker. Aunque el desarrollo se haya realizado en Windows, el stack puede ejecutarse en Linux, macOS o en una instancia EC2. Las diferencias entre sistemas operativos se documentan, especialmente en componentes de monitorización que dependen del host, como Node Exporter, cAdvisor o Telegraf.

La seguridad se ha planteado por capas: aplicación, base de datos, contenedores, red y operación. Ninguna medida es suficiente. El control de roles protege funciones internas, las consultas preparadas reducen riesgo de inyección SQL, los secretos deben estar fuera del repositorio y los servicios de administración no deben exponerse a Internet.

Tabla 9. Requisitos no funcionales

Fuente: elaboración propia.

5.8. Detalle de módulos funcionales

5.8.1. Módulo de autenticación y usuarios

El módulo de autenticación cubre registro, inicio de sesión, cierre de sesión y control de rol. El registro inserta datos personales y credenciales en tablas separadas. Esta separación permite que la información de acceso tenga su

propio ciclo de validación y que el perfil del usuario pueda evolucionar sin modificar la tabla de login. Es decir, la información del usuario se divide en dos tablas así nos aporta mayor seguridad y es más sencillo modificarlo sin afectar al usuario.

El inicio de sesión comprueba el nombre de usuario y valida la contraseña con `password_verify`. Al validarse, se guardan en sesión el identificador de usuario, el nombre de usuario y el rol. La regeneración del identificador de sesión reduce el riesgo de fijación de sesión, una medida básica pero importante en aplicaciones PHP.

El control de roles se centraliza en funciones auxiliares. Esta decisión evita repetir lógica en cada página y facilita auditorías: las páginas administrativas deben llamar a `isAdmin` antes de mostrar el contenido sensible. En una evolución futura se podría ampliar a un sistema de permisos más granular.

5.8.2. Módulo de citas

El módulo de citas es el núcleo funcional del sistema. Gestiona la disponibilidad horaria, las reservas de usuarios registrados y las reservas de invitados. La decisión de permitir invitados responde a un caso real: muchos clientes desean reservar rápido sin crear una cuenta, pero el taller necesita un nombre, correo y teléfono para poder contactar.

La interfaz muestra un calendario mensual y consulta por AJAX los tramos ocupados. El backend devuelve un mapa de fechas y horas reservadas. El navegador construye botones de horario y deshabilita los ocupados. Aunque esta validación mejora la experiencia, la validación definitiva siempre se repite en servidor antes de insertar la cita.

La API comprueba que la fecha no sea pasada, que el formato de hora sea válido y que el horario siga libre. Si otro usuario reserva el mismo tramo entre la consulta y el envío, la API responde con conflicto. Esta comprobación en servidor es esencial para evitar inconsistencias.

5.8.3. Módulo de contenidos: noticias y consejos

El módulo de noticias permite al taller publicar información como, campañas, avisos y novedades. Las noticias incluyen título, texto, fecha, autor e imagen.

En administración se valida que el título sea único y que el fichero subido sea una imagen permitida.

El módulo de consejos complementa las noticias con contenido técnico y o educativo. Puede utilizarse para publicar recomendaciones de mantenimiento, revisiones periódicas, presión de neumáticos o preparación del vehículo para ITV. Desde el punto de vista de negocio, este módulo mejora la comunicación con los clientes y aporta un valor añadido de la reserva de citas.

Estos módulos demuestran gestión de contenido en una aplicación propia. Aunque no son un CMS completo, cubren operaciones CRUD y subida o referencia de imágenes.

5.8.4. Módulo de administración

El panel de administración tiene funciones que no deben estar disponibles para usuarios normales. Desde este entorno se consultan estadísticas básicas, gestionan de usuarios, se revisa la agenda, se editan noticias y se cargan los consejos. Esta separación ayuda a explicar el control de acceso por roles y los riesgos de exposición.

La vista administrativa también sirve como punto de operación funcional. Un responsable del taller puede comprobar citas pendientes, editar datos de usuarios o publicar avisos sin acceder directamente a la base de datos. Esto evita operaciones peligrosas y reduce la necesidad de conocimientos técnicos en el día a día del negocio.

Desde la perspectiva de sistemas, el panel no sustituye a Grafana ni a Prometheus. La administración funcional se realiza en la aplicación; la administración técnica se realiza en herramientas de observabilidad. Mantener esta separación evita mezclar operaciones del negocio, como las que podría realizar un usuario y las operaciones de infraestructura que las realizaría el administrador de sistemas.

5.9. Procedimiento de despliegue detallado

El despliegue local comienza preparando variables de entorno. El administrador copia `.env.example` a `.env`, revisa contraseñas, puertos y parámetros de monitorización. Después ejecuta `"docker compose up -d --build"`. Docker

construye la imagen de la aplicación, descarga imágenes de terceros, crea redes, volúmenes y arranca servicios con healthchecks.

El primer arranque de MySQL ejecuta `database/database.sql` porque el volumen está vacío. Si el volumen ya existe, MySQL no vuelve a ejecutar los scripts de inicialización. Por ello se documenta la diferencia entre despliegue fresco, migración y restauración. Esta es una incidencia habitual en entornos Docker.

Tras levantar el stack, se verifica la aplicación web, el endpoint de métricas y los targets de Prometheus. En Grafana se comprueba que el datasource Prometheus está provisionado y que el dashboard principal carga datos. Si se activa el perfil de tráfico, se valida que el simulador, la UI y la aplicación comparten el mismo volumen de logs.

En AWS, el procedimiento cambia. La instancia EC2 se prepara con Docker, ya sea manualmente, mediante una AMI generada con Packer o mediante el user data de Amazon Linux 2023. El repositorio se clona en `/opt/taller_mecanico_asir`, se crea o revisa `.env`, se rotan secretos y se ejecuta `scripts/deploy_aws_docker.sh`. Las UI de monitorización se publican en puertos de navegador, pero el acceso debe quedar restringido por Security Group o por un proxy seguro.

Tabla 10. Checklist de despliegue

Fuente: elaboración propia.

5.9.1. Seguimiento del primer despliegue en EC2

Durante el primer arranque en EC2 no hay que asumir que Docker ya ha terminado. El user data todavía puede estar instalando paquetes, creando swap, descargando imágenes o construyendo contenedores. Por eso docker ps puede fallar, mostrar poca cosa o no reflejar el estado final hasta que el bootstrap termine.

Para ver cómo va cargando el despliegue se entra por SSH o Session Manager y se ejecuta: `cat /var/log/taller-ec2-bootstrap.log`. Si se quiere verlo en directo, el comando más cómodo es `sudo tail -f /var/log/taller-ec2-bootstrap.log`. El log muestra pasos numerados del bootstrap y, si algo falla, incluye diagnósticos de docker compose `ps -a` y logs de servicios.

El bootstrap añade ec2-user al grupo docker. Ese cambio no se aplica a una terminal que ya estaba abierta. Después de terminar el bootstrap hay que salir con “exit” y abrir una nueva sesión SSH, o ejecutar “newgrp Docker”. Hasta hacer eso, docker ps puede pedir sudo aunque el usuario ya esté añadido al grupo.

Las contraseñas de producción no son las de la documentación local. En EC2 el bootstrap o el administrador rotan los secretos en /opt/taller_mecanico_asir/.env. Para consultar la contraseña de Grafana se usa “cd /opt/taller_mecanico_asir” y después “grep GRAFANA .env”. Para revisar Prometheus se usa “grep PROMETHEUS .env.” en este compose Prometheus no trae usuario y contraseña propios, por lo que se protege con Security Group o proxy; si en producción se añade autenticación por proxy, su variable también debe quedar en .env. Para otros servicios se usa el mismo patrón: grep MYSQL .env para MySQL, grep SIMULATOR .env para el token del simulador, grep ALERTMANAGER .env para la UI de alertas o grep SMTP .env para correo.

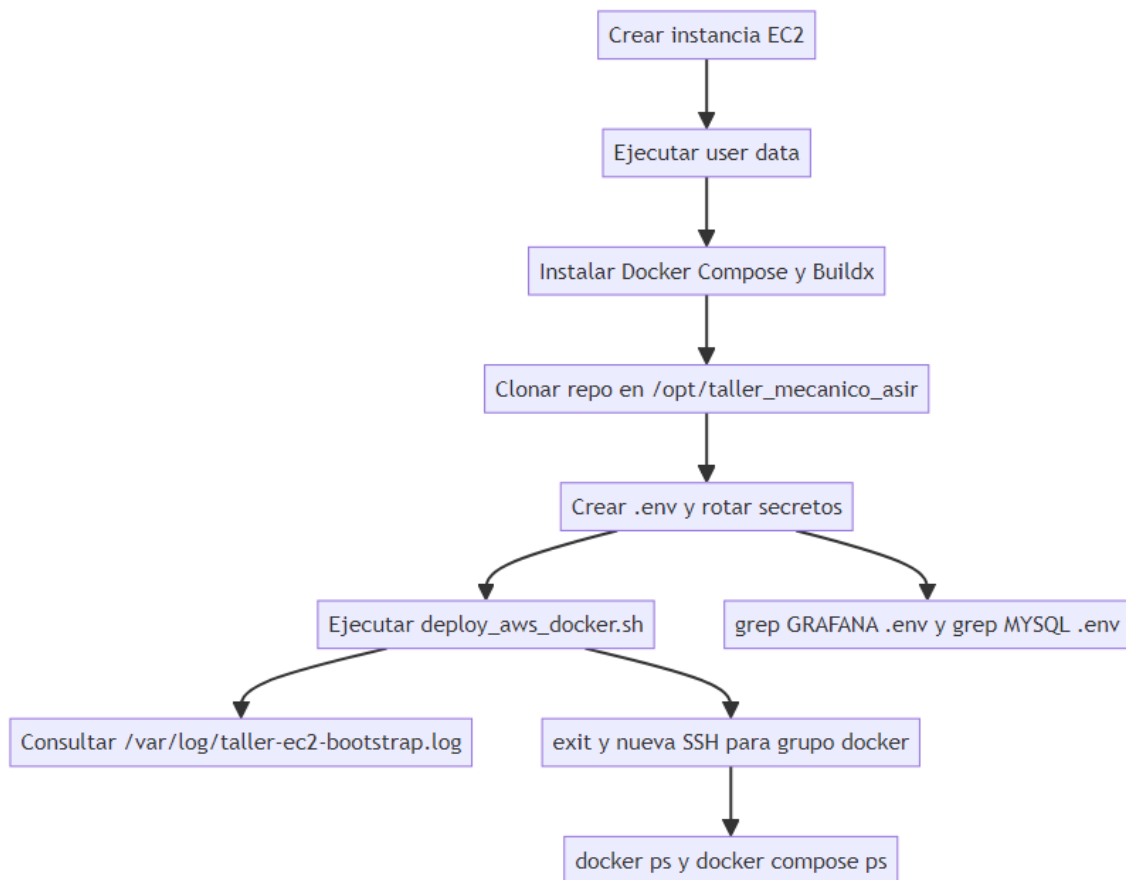


Figura 23. Flujo operativo del despliegue EC2 y consulta de credenciales.

Fuente: elaboración propia a partir de scripts/ec2-user-data-bootstrap.sh y scripts/deploy_aws_docker.sh.

5.10. Copias de seguridad y restauración

La base de datos es el activo crítico del sistema. Contiene a los usuarios, citas y contenidos publicados. En un entorno real, la pérdida de la base de datos implicaría la pérdida de la agenda y datos de clientes. Por ello en la memoria se propone backups SQL y snapshots del volumen EBS.

En despliegue Docker local se pueden realizar backups con mysqldump dentro del contenedor MySQL. En producción se recomienda automatizar el proceso, comprimir el resultado y copiarlo a almacenamiento externo cifrado. La copia debe probarse mediante restauración, no basta con comprobar que el fichero existe.

La restauración debe hacerse con el servicio MySQL en ejecución y con credenciales correctas. Si se restaura sobre una base ya inicializada, pueden aparecer conflictos de claves duplicadas. Para una restauración limpia, se recomienda recrear el volumen o importar sobre una base vacía. Esta diferencia se documenta ya que es crucial porque afecta directamente al procedimiento de recuperación ante desastre.

Los ficheros de imágenes subidos por administradores también deben respaldarse. En el compose de producción se almacenan en un volumen nombrado. Si solo se copia la base de datos y se pierden las imágenes, las noticias podrían aparecer con enlaces rotos. Por tanto, el plan de backup debe cubrir base de datos y assets persistentes.

5.11. Despliegues alternativos: Coolify y Dokploy

Además de AWS, el repositorio contempla despliegues en plataformas de tipo PaaS autogestionado como Coolify y Dokploy. Estas herramientas simplifican la publicación desde repositorio Git, gestión de dominios, HTTPS y despliegues Docker Compose. Son muy útiles para pequeños proyectos donde no se quiere operar toda la infraestructura de forma manual.

Coolify permite separar la aplicación y la monitorización en recursos distintos. Esta separación evita conflictos de puertos y facilita exponer Grafana, Prometheus o Alertmanager mediante rutas controladas. La documentación recomienda no publicar puertos fijos al host cuando la plataforma ya ofrece domains/routes.

Dokploy sigue una filosofía parecida: despliegue desde repositorio, compose específico y variables de entorno gestionadas desde un panel. En ambos casos se mantiene la necesidad de rotar secretos, proteger métricas y limitar acceso a herramientas administrativas.

La existencia de varias formas de despliegue aporta valor ya que demuestra que la aplicación no depende de una única plataforma. El administrador puede elegir entre un entorno local, un VPS/EC2 clásico o una plataforma de despliegue con panel, según coste, experiencia y necesidades de la empresa.

Tabla 11. Comparativa de escenarios de despliegue

Fuente: elaboración propia.

5.12. Operación diaria y mantenimiento

Una vez implantado el sistema, pasaremos a operación y mantenimiento. El administrador debe revisar dashboards, comprobar alertas, actualizar imágenes, rotar credenciales y validar backups. La documentación del proyecto incluye comandos de logs, reinicio de servicios y troubleshooting.

Las operaciones diarias pueden estructurarse en tres niveles. El primer nivel revisar disponibilidad: la aplicación responde, Prometheus extrae la información y Grafana muestra datos. El segundo nivel revisa el rendimiento: latencia, errores 5xx, conexiones de MySQL, CPU, memoria y disco. El tercer nivel revisa el negocio: número de citas, crecimiento de usuarios y actividad de contenidos.

La política de las nuevas versiones de la aplicación debe ser conservadora. En desarrollo se pueden probar versiones recientes, pero en producción es conveniente fijar tags concretos, validar cambios en un entorno de pruebas y aplicar actualizaciones con ventana de mantenimiento. Las actualizaciones de PHP, MySQL, Grafana y Prometheus deben revisarse por notas de seguridad.

La revisión de logs se complementa con las métricas. Estas indican tendencias y estados; los logs explican detalles de una petición o error. En el proyecto, los logs HTTP y de tiempos alimentan el exporter PHP, mientras que los logs de contenedores permiten investigar fallos de arranque, conexión o permisos.

Tabla 12. Plan de mantenimiento preventivo

Fuente: elaboración propia.

6. RECURSOS Y PRESUPUESTO

6.1. Recursos utilizados

Tabla 13. Recursos utilizados

Fuente: elaboración propia.

6.2. Presupuesto anual orientativo

El presupuesto se calcula para una pequeña empresa con una única instancia EC2 ejecutando la aplicación, base de datos y monitorización. Los costes son orientativos y pueden variar por región, tráfico, tipo de instancia y política vigente del proveedor. Para una primera implantación se considera suficiente una arquitectura sencilla; si el servicio crece, se recomienda la separación de la base de datos y monitorización.

Tabla 14. Presupuesto anual orientativo

Fuente: elaboración propia.

El coste podría reducirse usando instancias gratuitas o créditos educativos, pero para una empresa se ha preferido un cálculo conservador. No se incluye IVA ni posibles costes de soporte externo. Si se añade un Application Load Balancer, CloudWatch intensivo o RDS MySQL, el coste anual aumentaría, aunque también mejoraría la operación.

7. VALIDACIÓN Y PRUEBAS

7.1. Pruebas funcionales

Las pruebas funcionales se organizan por perfiles. Como visitante se comprueba la navegación, visualización de noticias, registro y reserva como invitado. Como usuario registrado se verifica login, actualización de perfil y visualización de citas propias. Como administrador se prueban las operaciones CRUD y la protección de accesos.

La reserva de citas se puede validar de diferentes formas: fecha pasada, hora ocupada, datos incompletos, usuario autenticado y cliente invitado. La API debe responder con códigos adecuados y mensajes comprensibles. La interfaz actualiza los horarios ocupados mediante fetch para evitar que el usuario seleccione tramos no disponibles.

Las pruebas de subida de imágenes incluyen archivos válidos y no válidos. Se comprueba extensión, tipo MIME, tamaño máximo y escritura en el directorio persistente. En Docker, el volumen de imágenes debe conservarse tras reiniciar contenedores.

7.2. Pruebas de despliegue y monitorización

La validación de un despliegue limpio consiste en eliminar volúmenes de desarrollo, reconstruir imágenes, levantar servicios y comprobar que MySQL esta healthy antes de que la aplicación web arranque. Esta verificación evita fallos típicos en los que Apache inicia antes de que la base de datos acepte conexiones.

Para Prometheus se revisan los targets y se confirma que php-app, mysqld-exporter, node-exporter y blackbox-exporter están en estado UP. En Grafana se comprueba que el datasource Prometheus se provisiona automáticamente y que el dashboard muestra métricas de aplicación, base de datos, sistema y negocio.

El simulador de tráfico se prueba con cargas breves, verificando que las líneas aparecen en logs/metrics.log y que Grafana diferencia source=app de

source=simulator. Este punto es relevante porque permite demostrar observabilidad bajo tráfico generado sin alterar el código de la aplicación.

7.3. Matriz de pruebas

La siguiente matriz resume pruebas recomendadas para la defensa y para una validación previa. Esta no sustituye a una suite automatizada completa, pero permite demostrar los flujos principales y detectar fallos críticos.

Tabla 15. Matriz de pruebas funcionales y técnicas

Fuente: elaboración propia.

7.4. Plan de demostración para la defensa

La exposición puede organizarse en tres bloques. En primer lugar, se presenta el problema del taller y la solución funcional: navegación, registro, login, cita y panel de administración. Este bloque debe ser breve.

En segundo lugar, se muestra la implantación: docker compose ps, servicios levantados, base de datos, volúmenes y redes. Conviene explicar por qué MySQL no debe exponerse y cómo el servicio web depende de healthchecks.

En tercer lugar, se demuestra observabilidad: endpoint de métricas, Prometheus targets, dashboard Grafana y simulador de tráfico. Una carga breve permite ver cómo cambian las gráficas, diferenciando tráfico real y simulado. Esta demostración aporta evidencia visual de que la monitorización no es decorativa.

Como cierre, se explican las mejoras futuras: RDS, CI/CD, HTTPS definitivo, backups en S3 y políticas RGPD.

Tabla 16. Guion de demostración

Fuente: elaboración propia.

8. CONCLUSIONES

8.1. Grado de consecución de objetivos

Tabla 17. Grado de consecución de objetivos

Fuente: elaboración propia.

8.2. Problemas encontrados

Tabla 18. Problemas encontrados y resolución

8.2.1. Dificultades técnicas (SMTP externo y CI/CD)

La integración de notificaciones por correo con un SMTP externo (Alertmanager) presentó fricción operativa: muchos proveedores obligan a STARTTLS en el puerto 587; en Amazon SES las credenciales SMTP no coinciden con las claves de acceso IAM y hace falta verificar remitente y destinatarios mientras la cuenta siga en sandbox. Alertmanager no ofrece un botón de prueba en su interfaz web, por lo que la validación exige disparar una alerta o usar el panel admin de la aplicación cuando el stack está levantado. Si faltan variables obligatorias, el endpoint puede arrancar en modo noop para no bloquear el despliegue, lo que a veces confunde hasta rellenar ALERT_EMAIL_TO, SMTP_SMARTHOST y SMTP_FROM.

Respecto al CI/CD con GitHub Actions, los workflows de verificación de builds añadían coste de mantenimiento (imágenes base, Buildx en runners, secretos para despliegue) y fallos intermitentes por red o límites de tiempo. Se ha retirado la integración con GitHub Actions; la comprobación de imágenes pasa a hacerse en local con docker compose build.

Fuente: elaboración propia.

8.3. Mejoras futuras

Tabla 19. Propuestas de mejora

Fuente: elaboración propia.

9. ANEXOS Y DOCUMENTOS COMPLEMENTARIOS

9.1. Índice de anexos

- Anexo A. Inventario de archivos principales del repositorio.
- Anexo B. Comandos de despliegue local y AWS.
- Anexo C. Variables de entorno relevantes.
- Anexo D. Fragmentos de configuración y observabilidad.
- Anexo E. Referencias y bibliografía.

Anexo A. Inventario de archivos principales

Tabla 20. Inventario de artefactos del repositorio

Fuente: elaboración propia.

Inventario actualizado por carpetas

La raíz contiene las páginas PHP visible para todo el mundo. La carpeta admin/ agrupa el panel (admin/index.php y CRUD en admin/usuarios.php, admin/citas.php, admin/noticias.php y admin/consejos.php); api contiene endpoints JSON; config centraliza el entorno y conexión a base de datos; includes contiene cabecera, pie y registro de métricas; database guarda esquema, migraciones y datos semilla.

La carpeta docker contiene entypoints, backup y las dos imágenes del simulador: el worker con Apache JMeter y la UI Apache/PHP. La carpeta monitoring contiene Prometheus, Grafana, Alertmanager, exporters, Telegraf y cAdvisor. La carpeta scripts agrupa despliegue AWS, verificaciones, herramientas de base de datos, scraping de noticias y ejecución de JMeter. La carpeta tests mantiene pruebas PHP de acceso y del simulador.

La documentación se reparte en README.md y docs. Las guías más importantes para operación son DOCKER_DEPLOYMENT.md para local, AWS_DOCKER_DEPLOYMENT.md para EC2, MONITORING_SETUP_GUIDE.md para observabilidad y TRAFFIC_SIMULATOR.md para JMeter. Los ficheros docker-compose.coolify*.yml y docker-compose.dokploy.yml documentan despliegues alternativos en plataformas con panel.

Anexo B. Comandos de despliegue

Comandos útiles en EC2 tras el bootstrap

```
cd /opt/taller_mecanico_asir
cat /var/log/taller-ec2-bootstrap.log
grep GRAFANA .env
grep MYSQL .env
grep PROMETHEUS .env
grep SIMULATOR .env
exit
```

```
docker ps
```

```
docker compose --env-file .env -f docker-compose.aws.yml ps -a
```

Estos comandos que se mostraron anteriormente en el apartado 5.4.1 sirven para comprobar que el despliegue ha terminado, encontrar contraseñas reales de producción y confirmar que la sesión SSH ya reconoce los permisos del grupo docker.

Despliegue local con Docker: copiar `.env.example` a `.env`, ajustar credenciales y ejecutar `docker compose up -d --build`. La aplicación queda disponible en el puerto configurado por `WEB_PORT` y Grafana en `GRAFANA_PORT`.

Despliegue con simulador: ejecutar `docker compose --profile traffic up -d` y acceder a la UI en el puerto configurado por `TRAFFIC_SIMULATOR_UI_PORT`. Antes de usarlo debe cambiarse `SIMULATOR_CONTROL_TOKEN`.

Despliegue AWS: crear instancia EC2, usar el `user data scripts/ec2-user-data-bootstrap.sh` o una AMI preparada, clonar el repositorio en `/opt/taller_mecanico_asir`, revisar `.env`, rotar secretos y ejecutar `./scripts/deploy_aws_docker.sh`. El bootstrap escribe su avance en `/var/log/taller-ec2-bootstrap.log` y el script imprime al final las URLs de navegador de Grafana, Prometheus, Alertmanager y la UI del simulador.

Anexo C. Variables de entorno relevantes

Tabla 21. Variables principales de configuración

Fuente: elaboración propia.

Anexo D. Fragmentos de configuración y observabilidad

Los siguientes fragmentos resumen decisiones relevantes. Se recomienda consultar los archivos originales del repositorio para ver el contenido completo y actualizado.

Docker Compose local define `web`, `mysql`, `prometheus`, `grafana`, `alertmanager`, `node-exporter`, `mysqld-exporter`, `blackbox-exporter`, `cadvisor`, `telegraf` y servicios opcionales de simulación. Los volúmenes conservan datos, logs e imágenes, y las redes separan aplicación y monitorización.

Prometheus obtiene métricas de /metrics.php, exporters y probes blackbox. Las reglas de alertas incluyen caída de métricas de aplicación, errores 5xx, latencia elevada, base de datos no saludable, conexiones MySQL altas, recursos de sistema y fallo de probes HTTP.

10. REFERENCIAS Y BIBLIOGRAFÍA

Amazon Web Services. Documentación oficial de Amazon EC2, Amazon EBS, Elastic Load Balancing y AWS Systems Manager.

<https://docs.aws.amazon.com/>

Apache Software Foundation. Apache HTTP Server Documentation.

<https://httpd.apache.org/docs/>

Docker Inc. Docker Documentation y Docker Compose Documentation.

<https://docs.docker.com/>

Grafana Labs. Grafana Documentation. <https://grafana.com/docs/>

HashiCorp. Packer Documentation.

<https://developer.hashicorp.com/packer/docs>

MySQL. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/>

PHP Group. PHP Manual: PDO, sessions, password hashing and security functions. <https://www.php.net/docs.php>

Prometheus Authors. Prometheus, Alertmanager, Node Exporter y Blackbox Exporter Documentation. <https://prometheus.io/docs/>

Repositorio del proyecto Taller Mecánico ASIR. Documentación interna: README.md, docs/DOCKER_DEPLOYMENT.md, docs/AWS_DOCKER_DEPLOYMENT.md, docs/MONITORING_SETUP_GUIDE.md y docs/TRAFFIC_SIMULATOR.md.

Módulo	Aplicación en el PFC
Implantación de Aplicaciones Web	Despliegue de una aplicación PHP sobre Apache, gestión de sesiones, formularios, páginas públicas y privadas, estructura de rutas y publicación mediante Docker.
Gestión de Bases de Datos	Diseño de tablas MySQL, claves primarias y foráneas, relaciones 1:1 y 1:N, índices, restricciones de unicidad y persistencia de usuarios, citas, noticias y consejos.

Módulo	Aplicación en el PFC
Administración de Sistemas Gestores de Bases de Datos	Configuración de MySQL 8.0, inicialización mediante script SQL, usuarios de aplicación, acceso por variables de entorno, backups y posibilidad de migración futura a RDS O sistema de bases de datos relacional de Amazon.
Servicios de Red e Internet	Exposición de servicios HTTP, uso de puertos, redes Docker internas, resolución por nombres de servicio, rutas de monitorización y publicación mediante dominio.
Administración de Sistemas Operativos	Gestión de contenedores, volúmenes, logs, healthchecks, scripts de despliegue, permisos y resolución de incidencias en entornos Windows/Linux.
Seguridad y Alta Disponibilidad	Autenticación por roles, contraseñas cifradas, sesiones, Cross-Site Request Forgery en formularios críticos, sentencias preparadas, sanitización, control de puertos y estrategia de alertas.
Planificación y Administración de Redes	Separación de redes de aplicación y monitorización, definición de Security Groups en AWS, acceso restringido a Prometheus, Grafana, Alertmanager y MySQL.
Lenguajes de Marcas y Sistemas de Gestión de Información	HTML5, CSS3, JavaScript, documentación Markdown, configuración YAML de Docker Compose y JSON de dashboards Grafana.
Empresa e Iniciativa Emprendedora	Análisis de una necesidad de negocio realista, digitalización de procesos de taller, presupuesto de implantación y evaluación de costes de explotación.
Inglés Técnico	Uso de documentación oficial de Docker, AWS, Prometheus, Grafana, PHP y MySQL, así como nomenclatura técnica habitual en herramientas cloud y de observabilidad.
Perfil	Funciones principales
Visitante	Consultar página de inicio y noticias, registrarse, iniciar sesión y solicitar una cita como invitado con datos de contacto.

Módulo		Aplicación en el PFC	
Usuario registrado		Crear reservas, consultar citas propias, editar perfil, cambiar contraseña y mantener sus datos personales.	
Administrador		Gestionar usuarios, citas, noticias, consejos, comprobar estadísticas básicas y acceder a herramientas de monitorización.	
Operador técnico		Desplegar el sistema, revisar logs, comprobar métricas, validar healthchecks, ejecutar simulador de tráfico y restaurar backups.	
Alternativa	Ventajas	Limitaciones frente al PFC	
Agenda manual o hoja de cálculo	Coste inicial bajo y aprendizaje rápido.	No hay acceso web, control de roles, trazabilidad, validaciones ni monitorización.	
Google Calendar compartido	Permite organizar citas y avisos básicos.	No integra noticias, perfiles, panel de administración ni métricas de negocio propias.	
SaaS de gestión de talleres	Producto maduro con soporte comercial.	Coste recurrente y dependencia de proveedor	
WordPress con plugins de reservas	Rápido de publicar y con ecosistema amplio.	Mas fácil de atacar por plugins, menor control del backend y de la arquitectura.	
Solución propuesta PHP/MySQL Docker	Control completo, despliegue reproducible y monitorización integrada.	Requiere administración técnica y mantenimiento.	
Semana	Tareas realizadas	Horas	
Lunes 03/03/26 - Viernes 07/03/26	Análisis de requisitos, definición de escenario de taller, revisión de los módulos ASIR implicados y estructura inicial del repositorio.	3	
Lunes 10/03/26 - Viernes 14/03/26	Diseño del modelo de datos, tablas principales, relaciones, restricciones y usuarios iniciales.	5	

Módulo		Aplicación en el PFC	
Lunes 17/03/26 - Viernes 21/03/26		Desarrollo de autenticación, registro, login, sesiones, roles y estructura de cabecera/pie.	6
Lunes 24/03/26 - Viernes 28/03/26		Implementación de gestión de citas, calendario, API JSON, validación de horarios y reservas de invitados.	7
Lunes 31/03/26 - Viernes 04/04/26		Implementación de noticias, consejos, subida de imágenes y panel de administración.	5
Lunes 07/04/26 - Viernes 11/04/26		Creación de los contenedores en Docker, variables de entorno, volúmenes, redes, healthchecks e inicialización de MySQL.	7
Lunes 14/04/26 - Viernes 18/04/26		Monitorización con Prometheus, Grafana, exporters, endpoint de métricas PHP, alertas y dashboards.	6
Lunes 21/04/26 - Viernes 25/04/26		Simulador de tráfico, pruebas de carga controladas, documentación de despliegue y revisión de seguridad.	5
Lunes 28/04/26 - Viernes 02/05/26		Redacción de memoria, anexos, presupuesto, conclusiones y preparación de defensa.	6
Total		Planificación, implantación y documentación del PFC.	50
Tabla	Propósito	Relaciones	
users_data	Datos personales y de contacto.	Relación 1:1 con users_login y 1:N con citas, noticias y consejos.	
users_login	Credenciales, usuario y rol.	Clave foránea idUser con eliminación en cascada.	

Módulo		Aplicación en el PFC
citas	Reservas con fecha, hora, motivo y datos de invitado.	Puede vincularse a users_data o registrar datos de invitado.
noticias	Publicaciones del taller con imagen y texto.	Cada noticia se asocia al usuario administrador creador.
consejos	Consejos técnicos o contenido informativo.	Cada consejo se asocia al usuario que lo creó.
Servicio	Imagen o build	Función
web	Build desde Dockerfile	Apache + PHP 8.2, aplicación web y endpoint /metrics.php.
mysql	mysql:8.0	Base de datos relacional con volumen persistente e inicialización SQL.
prometheus	prom/prometheus:v3.11.3	Scraping de métricas y evaluación de reglas de alerta.
grafana	grafana/grafana:13.0.1	Dashboards técnicos y de negocio provisionados desde archivos.
alertmanager	prom/alertmanager:v0.32.1	Agrupación y envío de alertas por correo SMTP.
node-exporter	prom/node-exporter:v1.11.1	Métricas del sistema operativo anfitrión.
mysqld-exporter	prom/mysqld-exporter:v0.19.0	Métricas de rendimiento de MySQL.
blackbox-exporter	prom/blackbox-exporter:v0.26.0	Probes HTTP sintéticos sobre rutas críticas.
traffic-simulator	Build desde docker/traffic-simulator	Generación de tráfico HTTP contra la aplicación.
traffic-simulator-ui	Build desde docker/traffic-simulator-ui	Interfaz web separada para arrancar/parar simulaciones.

Módulo		Aplicación en el PFC	
Componente	Exposición recomendada	Justificación	
Aplicación web	80/443 mediante proxy, ALB o Cloudflare	Servicio público para usuarios y clientes del taller.	
MySQL	Solo red Docker interna	Evita exposición directa de base de datos.	
Prometheus	IP_PUBLICA:9090, restringido por Security Group	Interfaz accesible desde navegador; no tiene autenticación fuerte propia, así que debe restringirse por Security Group o proxy seguro.	
Grafana	IP_PUBLICA:3000, restringido por Security Group	Panel de operación accesible desde navegador si el Security Group permite el puerto; en producción se recomienda limitarlo a la IP del administrador.	
Alertmanager	IP_PUBLICA:9093, restringido por Security Group	Interfaz de alertas accesible desde navegador si se abre el puerto; debe protegerse igual que Prometheus.	
Exporters	Solo red interna	Deben ser consumidos por Prometheus, no por Internet.	
Área	Métricas o alertas	Utilidad	
Aplicación	Requests por método, código HTTP, fuente app/simulator y percentiles de respuesta.	Detectar errores, latencia y efecto de pruebas de carga.	
Negocio	Usuarios totales, citas totales, citas futuras y pasadas, noticias y consejos.	Medir actividad del taller.	

Módulo		Aplicación en el PFC
Base de datos	Conexiones, salud de conexión, consultas, slow queries y disponibilidad del exporter.	Detectar saturación o caída de MySQL.
Sistema	CPU, memoria, disco y red.	Vigilar recursos de la instancia o host Docker.
Blackbox	Probe success sobre rutas /, /login.php, /citaciones.php y /metrics.php.	Confirmar disponibilidad desde fuera del proceso PHP.
Alertmanager	Agrupación por severidad y envío SMTP.	Reducir ruido y notificar incidencias críticas.
Categoría	Decisión de diseño	Evidencia en el proyecto
Disponibilidad	Healthchecks y probes HTTP.	docker-compose.yml, docker-compose.aws.yml, blackbox.yml.
Mantenibilidad	Documentación técnica y ficheros de configuración por entorno.	docs/, .env.example, .env.aws.example.
Portabilidad	Contenedores Docker y Compose.	Dockerfile, docker-compose*.yml.
Seguridad	Roles, sesiones, secretos por entorno y puertos restringidos.	includes/functions.php, config/database.php, compose AWS.
Observabilidad	Métricas, logs, dashboards y alertas.	monitoring/, includes/metrics_logger.php.
Escalabilidad inicial	Separación de servicios y propuesta futura RDS/CI-CD.	docs/AWS_DOCKER_DEPLOYMENT.md y conclusiones.
Paso	Comprobación	Resultado esperado
Preparar .env	Contraseñas, puertos, SMTP y token del simulador.	Sin secretos por defecto en producción.

Módulo		Aplicación en el PFC	
Construir imágenes	docker compose build o up -d --build.	Imagen web construida sin errores.	
Arrancar MySQL	Healthcheck mysqladmin ping.	Estado healthy antes de web.	
Arrancar web	Petición HTTP a /.	Respuesta 200 o página inicial visible.	
Validar métricas	Acceder a /metrics.php.	Formato Prometheus sin HTML ni errores.	
Validar Prometheus	Targets en estado UP.	Scraping correcto de app y exporters.	
Validar Grafana	Datasource y dashboard.	Paneles con datos.	
Validar alertas	Reglas cargadas y Alertmanager accesible.	Alertas agrupadas y listas para SMTP.	
Escenario	Ventaja principal	Riesgo o limitación	
Docker local	Rápido para desarrollo y pruebas completas.	No es una exposición pública profesional.	
AWS EC2 + Docker	Control completo de instancia, red, disco y seguridad.	Mayor responsabilidad operativa.	
Coolify	Gestión cómoda de dominios, rutas y despliegues.	Dependencia de la plataforma instalada.	
Dokploy	Despliegue Git/Compose con panel y dominios.	Requiere configurar correctamente secretos y persistencia.	
XAMPP local	Sencillo para pruebas PHP/MySQL en Windows.	No incluye observabilidad ni reproduce producción.	
Periodicidad	Tarea	Herramienta	
Diaria	Comprobar disponibilidad, errores HTTP y alertas activas.	Grafana, Prometheus, Alertmanager.	

Módulo		Aplicación en el PFC
Semanal	Revisar espacio en disco, crecimiento de logs y estado de backups.	Node Exporter, Docker, scripts de backup.
Mensual	Aplicar actualizaciones menores probadas y revisar contraseñas expuestas.	Docker Compose, gestor de secretos.
Trimestral	Probar restauración de backup en entorno aislado.	MySQL, volúmenes Docker, S3 futuro.
Anual	Revisar coste cloud, dominio, certificados y arquitectura.	AWS Billing, documentación interna.
Recurso	Uso en el proyecto	Licencia o coste
PHP 8.2 y Apache	Ejecución de la aplicación web.	Software libre.
MySQL 8.0	Persistencia de datos.	Software libre.
Docker y Docker Compose	Contenerización y orquestación local/cloud.	Software libre con Docker Desktop según condiciones de uso.
Prometheus	Recopilación de métricas y reglas de alerta.	Software libre.
Grafana	Visualización de métricas.	Edición OSS o Cloud según despliegue; en el proyecto se usa contenedor local.
Alertmanager	Envío y agrupación de alertas.	Software libre.
AWS EC2 y EBS	Infraestructura cloud propuesta para producción.	Servicio de pago por uso.
Packer	Construcción de AMI base con Docker.	Software libre.
Git/GitHub	Control de versiones y almacenamiento del repositorio.	Cuenta gratuita o educativa.

PFC

Módulo		Aplicación en el PFC		
Concepto	Cantidad	Coste unitario estimado	Coste anual	
EC2 t3.small 24/7	12 meses	25 EUR/mes	300 EUR	
EBS gp3 cifrado 50 GiB	12 meses	5 EUR/mes	60 EUR	
Dominio propio	1 año	12 EUR/año	12 EUR	
Backups/snapshots	12 meses	5 EUR/mes	60 EUR	
Correo SMTP para alertas	12 meses	0-10 EUR/mes	60 EUR	
Implantación inicial	10 horas	25 EUR/hora	250 EUR	
Mantenimiento preventivo	12 horas/año	25 EUR/hora	300 EUR	
Contingencia 10%	1	Sobre subtotal	104 EUR	
Total estimado	-	-	1.146 EUR	

Área	Prueba	Resultado esperado
Registro	Crear usuario con correo válido, teléfono español y código postal de 5 dígitos.	Usuario creado y redirección a login.
Registro	Intentar registrar correo duplicado.	Mensaje de error sin insertar duplicados.
Login	Acceder con credenciales correctas.	Sesión iniciada y regeneración de ID.
Login	Acceder con contraseña incorrecta.	Mensaje de error y sin sesión.
Roles	Usuario normal intenta entrar en administración.	Redirección o bloqueo.
Citas	Consultar disponibilidad de un mes.	JSON con horarios ocupados.

Módulo		Aplicación en el PFC	
Citas	Reservar hora libre como invitado.	Cita creada con datos de contacto.	
Citas	Reservar hora ya ocupada.	Respuesta de conflicto.	
Noticias	Subir imagen JPG/PNG válida.	Noticia publicada y fichero persistente.	
Noticias	Subir fichero no permitido.	Error de validación.	
Docker	Arranque limpio con volumen vacío.	MySQL inicializa esquema y web responde.	
Métricas	Acceder a /metrics.php.	Métricas Prometheus en texto plano.	
Prometheus	Revisar targets.	Targets principales en UP.	
Grafana	Abrir dashboard principal.	Paneles con datos.	
Alertas	Simular caída de endpoint.	Regla de alerta activa tras el tiempo definido.	
Simulador	Ejecutar carga corta.	Aumentan métricas con source=simulator.	
Minuto	Contenido	Evidencia	
0-5	Contexto, objetivos y módulos ASIR implicados.	Portada, introducción y tabla de módulos.	
5-12	Aplicación: login, citas, noticias y administración.	Navegador con flujos principales.	
12-18	Infraestructura Docker y base de datos.	docker compose ps, esquema y volúmenes.	
18-24	Monitorización y simulador.	Prometheus, Grafana y carga sintética.	

Módulo		Aplicación en el PFC	
24-28	AWS, seguridad y presupuesto.	Diagrama de despliegue y tabla de costes.	
28-30	Conclusiones y mejoras.	Tabla de objetivos y evolución propuesta.	
Objetivo		Grado	Observaciones
Aplicación web funcional para taller		Alto	Registro, login, citas, noticias, consejos y administración.
Modelo de datos relacional		Alto	Tablas normalizadas, claves foráneas, índices y datos iniciales.
Despliegue reproducible		Alto	Docker Compose local, variantes AWS/Coolify/Dokploy y documentación asociada.
Monitorización integrada		Alto	Prometheus, Grafana, Alertmanager, exporters y blackbox.
Simulación de tráfico		Medio-Alto	Worker y UI separados; requiere entorno Docker y configuración segura de token.
Preparación para producción		Medio	Existe compose AWS, pero faltaría desplegar dominio real, TLS definitivo y backups automáticos en S3.
Problema		Impacto	Solución aplicada o propuesta
Orden de arranque entre web y MySQL		La aplicación podía iniciar antes de que la base de datos estuviera lista.	Healthcheck de MySQL y dependencia condition: service_healthy.
Rutas del exporter PHP al copiarse a la imagen		El endpoint de métricas podía fallar si las rutas no coincidían.	Ajuste de rutas y validación de /metrics.php.

Módulo	Aplicación en el PFC	
Compatibilidad de Node Exporter en Windows	Algunos montajes Linux no existen en Windows.	Documentación de limitaciones y configuración compatible en desarrollo.
Exposición de herramientas de monitorización	Riesgo si Grafana/Prometheus quedan públicos sin control.	En AWS se publican solo en loopback y se accede por túnel o proxy seguro.
Simulador de tráfico potencialmente abusivo	Podría usarse contra URLs externas sin autorización.	Token, confirmación de externos, bloqueo de privados y SSL verify por defecto.
Mejora	Recursos necesarios	Beneficio esperado
Migrar MySQL a Amazon RDS	Instancia RDS, ajustes de variables y política de backups.	Menor carga operativa y backups gestionados.
Automatizar CI/CD con GitHub Actions	Runner, secretos, SSH/SSM o CodeDeploy.	Despliegues más repetibles.
HTTPS definitivo con ALB/ACM o Caddy	Dominio, certificado y configuración de proxy.	Confidencialidad y cumplimiento básico para datos personales.
Backups en S3 con retención	Bucket cifrado, lifecycle policy y script programado.	Recuperación ante pérdida de datos.
Notificaciones de cita por email	Proveedor SMTP transaccional y plantillas.	Mejor experiencia del cliente.
Calendario avanzado	Desarrollo adicional de interfaz y reglas de negocio.	Horarios especiales, festivos y cancelaciones.
Endurecimiento de la protección de datos	Política de privacidad, retención, exportación y borrado.	Mayor adecuación legal al tratar datos personales.

Módulo	Aplicación en el PFC	
Ruta	Descripción	
README.md	Descripción general, funcionalidades, credenciales de desarrollo y estado del proyecto.	
database/database.sql	Script de creación de base de datos, tablas, índices y usuario admin inicial.	
config/database.php	Conexión de PHP con la base de datos y soporte de variables de entorno para local, Docker y proveedores.	
includes/functions.php	Funciones comunes de sesión, roles, validación y ficheros.	
api/citas_api.php	API JSON para disponibilidad y creación de citas.	
docker-compose.yml	Stack local completo con aplicación, base de datos, monitorización y simulador opcional.	
docker-compose.aws.yml	Stack orientado a producción en EC2.	
monitoring/prometheus/prometheus.yml	Configuración de scrpts para Prometheus.	
monitoring/prometheus/alerts.yml	Reglas de alertas técnicas y de aplicación.	
monitoring/grafana/dashboards/taller-mecanico-dashboard.json	Dashboard principal de Grafana.	
docs/AWS_DOCKER_DEPLOYMENT.md	Guía de despliegue AWS con EC2, Docker Compose y Packer.	
docs/TRAFFIC_SIMULATOR.md	Documentación del simulador de tráfico y su UI separada.	
admin/	Panel de administración: admin/index.php y CRUD en admin/usuarios.php, admin/citas.php, admin/noticias.php y admin/consejos.php.	
tests/	Pruebas ad hoc en PHP (sin PHPUnit): citas_api, simulador de tráfico y smoke de sesión.	

Módulo	Aplicación en el PFC	
scripts/verify_local.ps1	Script PowerShell de verificación local: compose, JSON de Grafana, lint PHP y pruebas ad hoc.	
CONTEXT.md	Glosario de dominio (lenguaje ubicuo) del proyecto.	
Variable		Uso
DB_HOST, DB_NAME, DB_USER, DB_PASS		Conexión de la aplicación PHP a la base de datos.
MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD		Inicialización y credenciales de MySQL en Docker.
APP_ENV, APP_DEBUG		Control de entorno y salida de errores.
GRAFANA_ADMIN_USER, GRAFANA_ADMIN_PASSWORD		Credenciales iniciales de Grafana.
ALERT_EMAIL_TO, SMTP_*		Configuración de correo para Alertmanager.
WEB_PORT, PROMETHEUS_PORT, GRAFANA_PORT, WEB_HOST_PORT, PROMETHEUS_HOST_PORT, GRAFANA_HOST_PORT, ALERTMANAGER_HOST_PORT, TRAFFIC_SIMULATOR_UI_HOST_PORT		Puertos publicados en local y en AWS. En AWS se abren desde el Security Group y se recomienda restringir MONITORING_SG_CIDR a la IP del administrador.
SIMULATOR_CONTROL_TOKEN		Token de control del simulador de tráfico. En EC2 se genera o se define en /opt/taller_mecanico_asir/.env y no se muestra al navegador.
SIM_EXTERNAL_TARGETS_ENABLED, SIM_SSL_VERIFY		Política de seguridad del simulador.